



# Python Programming

ITC101

**Dr. Abhishek Jain**  
Assistant Professor,  
Department of Electronics Engg.



# Syllabus

| UNITs | Descriptions                                                                                                                                                                                                                                                                                                                                                  |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| I     | Introduction to computer science, algorithms, data representation in computers, hardware, software and operating system. Installation of python- interactive shell, IDLE, saving, editing, and running a script. The concepts of datatypes: variables, immutable variables, numerical types, operators, expressions, Indentation and comments in the program. |
| II    | Conditional Statements- Conditions, Boolean Logic, Logical operators and Ranges. Control Statements- Break, Continue and Pass. Flow Control-if, if-else, nested if-else, Loop statements- for loop, while loop, Nested loops.                                                                                                                                 |
| III   | String: subscript operator, indexing, slicing a string; strings and number system: converting strings to numbers and vice versa. Strings and text files, manipulating files and directories, os and sys modules, text files: reading/writing text and numbers from/to a file, creating and reading a formatted file (csv or tab-separated).                   |
| IV    | Lists, tuples, and dictionaries. Basic list operators, replacing, inserting, removing an element, searching and sorting lists, dictionary literals, adding and removing keys, accessing and replacing values, traversing dictionaries.                                                                                                                        |
| V     | Classes and OOP: Classes, objects, attributes and methods, defining classes, design with classes, Inheritance, Overloading, Overriding, and Data hiding. Exception: Exception Handling, except clause, Try finally clause. User Defined Exceptions.                                                                                                           |

## Text Books-

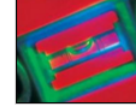
- Python Programming, R. NageshwarRao, Wiley India
- Think Python: Allen B. Downey, O'R



# Introduction to Computer Science

- ▶ computer science is the study of *computation* (more than just machinery)
  - ▶ it involves all aspects of problem solving, including
    - ▶ the design and analysis of algorithms
    - ▶ the formalization of algorithms as programs
    - ▶ the development of computational devices for executing programs
    - ▶ the theoretical study of the power and limitations of computing
- ▶ Computer science represents a rigorous approach to understanding complex phenomena and problem solving

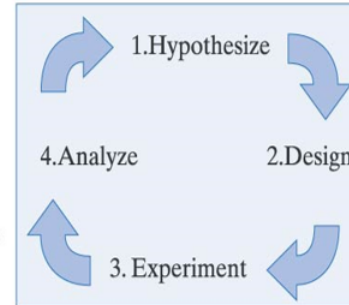
# Scientific Method



the process developed by the scientific community for examining observations and events is known as the *scientific method*

## The Scientific Method:

1. Formulate a hypothesis that explains an observed behavior.
2. Design an experiment to test your hypothesis.
3. Conduct the experiment.
4. Analyze the results of the experiment — if the results do not support the hypothesis, revise and repeat.



many activities carried out by computer scientists follow the scientific method

- e.g., designing and implementing a large database system requires hypothesizing about its behavior under various conditioning, experimenting to test those hypotheses, analyzing the results, and possibly redesigning
- e.g., debugging a complex program requires forming hypotheses about where an error might be occurring, experimenting to test those hypotheses, analyzing the results, and fixing the bugs





# Artificial Science

The distinction between computer science and natural sciences like biology, chemistry, and physics is the type of systems being studied

- natural sciences study naturally occurring phenomena and attempt to extract underlying laws of nature
- computer science studies human-made constructs: programs, computers, and computational modes

Herbert Simon coined the phrase "artificial science" to distinguish computer science from the natural sciences

In Europe, computer science is commonly called "Informatics"

- emphasizes the role of information processing as opposed to machinery

The term "Algorithmics" has also been proposed

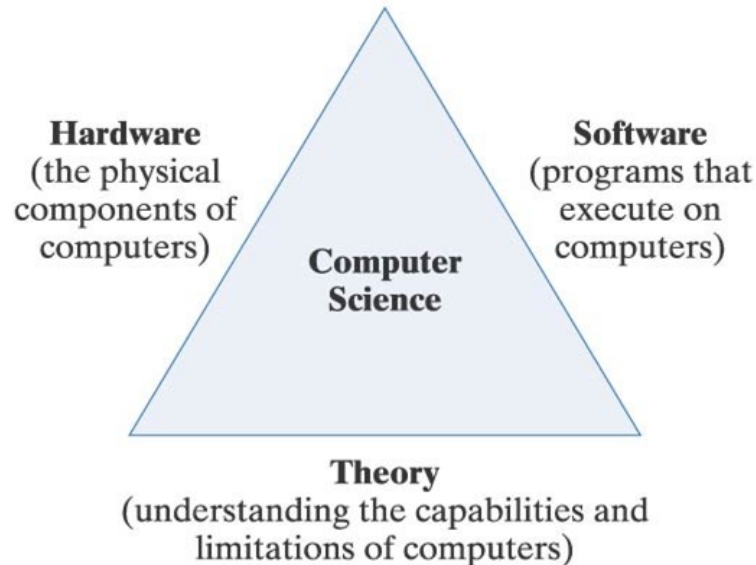
- emphasizes the role of algorithms and problem solving

Other related fields study computation from different perspectives

- *computer engineering* focuses on the design and construction of computers
- *information systems management* focuses on business applications

# Computer Science Themes

- ▶ since computation encompasses many different types of activities, computer science research is often difficult to classify
  - ▶ three recurring themes define the discipline



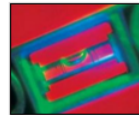
# Hardware



- ▶ *hardware* refers to the physical components of a computer and its supporting devices
- ▶ most modern computers implement the von Neumann architecture
  - ▶ CPU + memory + input/output devices
- ▶ ongoing research seeks to improve hardware design and organization
  - ▶ *circuit designers* create smaller, faster, more energy-efficient chips
  - ▶ *microchip manufacturers* seek to miniaturize and streamline production
  - ▶ *systems architects* research methods to increase *throughput* (the amount of work done in a given time period)
    - ▶ e.g., parallel processing – splitting the computation across multiple CPUs
    - ▶ e.g., networking – connecting computers to share information and work

# Software

---



*software* refers to the programs that execute on computers

## 3 basic software categories

1. *systems software*: programs that directly control the execution of hardware components (e.g., operating systems)
2. *development software*: programs that are used as tools in the development of other programs (e.g. Microsoft.NET, Java SDK)
3. *applications software*: all other programs, which perform a wide variety of tasks (e.g., web browsers, word processors, games)

many careers in computer science are related to the design, development, testing, and maintenance of software

- *language designers* develop and extend programming languages for easier and more efficient solutions
- *programmers* design and code algorithms for execution on a computer
- *systems analysts* analyze program designs and manage development

| Subfields of Computer Science <sup>1</sup>  |                                                                                                                                                                    |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Algorithms and Data Structures</b>       | The study of methods for solving problems, designing and analyzing algorithms, and effectively using data structures in software systems.                          |
| <b>Architecture</b>                         | The design and implementation of computing technology, including the integration of effective hardware systems and the development of new manufacturing methods.   |
| <b>Operating Systems and Networks</b>       | The design and development of software and hardware systems for managing the components of a computer or network of communicating computers.                       |
| <b>Software Engineering</b>                 | The development and application of methodologies for designing, implementing, testing, and maintaining software systems.                                           |
| <b>Artificial Intelligence and Robotics</b> | The study and development of software and hardware systems that solve complex problems through seemingly “intelligent” behavior.                                   |
| <b>Programming Languages</b>                | The design and implementation of languages that allow programmers to express algorithms so that they are executable on computers.                                  |
| <b>Databases and Information Retrieval</b>  | The organization and efficient management of large collections of data, including the development of methods for searching and recognizing patterns in the data.   |
| <b>Graphics</b>                             | The design of software and hardware systems for representing physical and conceptual objects visually, such as with images, video, or three-dimensional holograms. |
| <b>Human–Computer Interaction</b>           | The design, implementation, and testing of interfaces that allow users to interact more effectively with computing technology.                                     |
| <b>Computational Science</b>                | Explorations in science and engineering that utilize high-performance computing, such as modeling complex systems or simulating experimental conditions.           |
| <b>Organizational Informatics</b>           | The development and study of management processes and information systems that support technology workers and organizations.                                       |
| <b>Bioinformatics</b>                       | The application of computing methodologies and information structures to biological research, such as the characterization and analysis of the human genome.       |

# Subfields of Computer Science

computer science can be divided into subfields

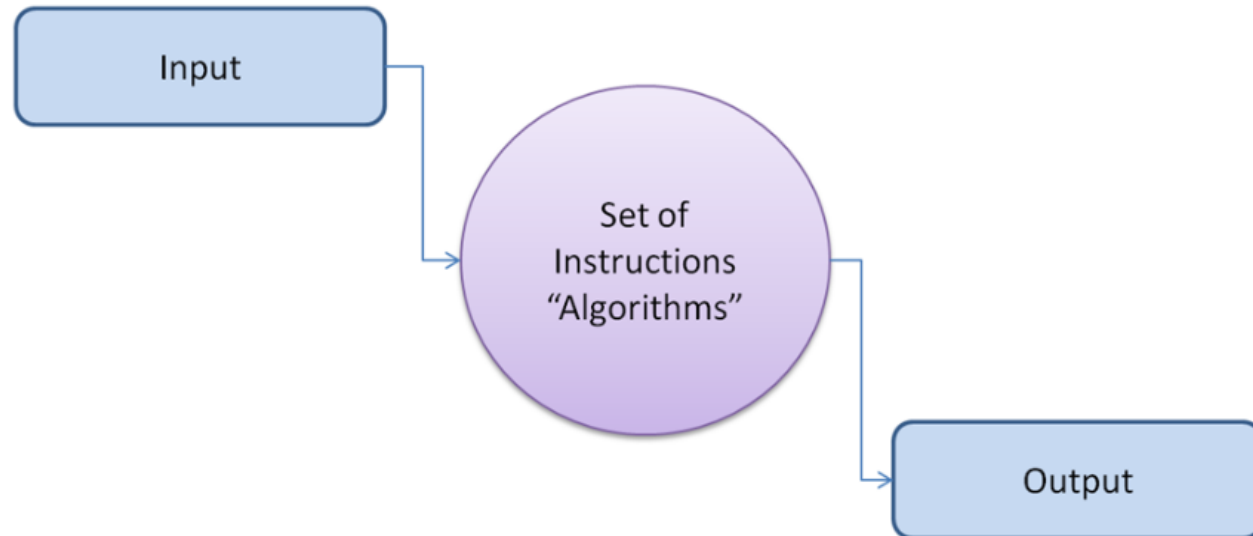
- each subfield takes a unique approach to computation
- however the common themes of computer science (hardware, software, and theory) influence every subfield



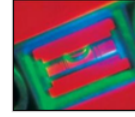
## What is an Algorithm?

**An Algorithm** is a step-by-step series of instructions to perform a certain task or specified actions. Task can vary depending on the situation; it can be as simple as adding two numbers or as complex as applying artificial intelligence. Talk about any field, be it computer science, databases, networking, security, or *operating system algorithms* form the fundamental aspect of every field. Algorithms are created to simplify complex tasks into easy and effective steps.

### What is Algorithm in Programming?



# Architecture



subfield concerned with methods of organizing hardware components into efficient, reliable systems

application: *parallel processing*

- multiple processors can sometimes be utilized to share the computational load
- there are costs associated with coordinating the processors and dividing the work, so not well suited for all tasks
- understanding when parallel processing can be used effectively is a common task for computer architects
  
- e.g., high-end Web Servers utilize multiple processors
  - ▣ can service multiple requests simultaneously by distributing the load among the processors
  
- Deep Blue, IBM's chess playing computer, contained 32 general purpose processors and 512 special-purpose chess processors
  - ▣ the processors worked in parallel to evaluate chess moves (could generate and evaluate 200 million chess moves per second)
  - ▣ in 1997, Deep Blue became the first computer to beat a world champion in a chess tournament

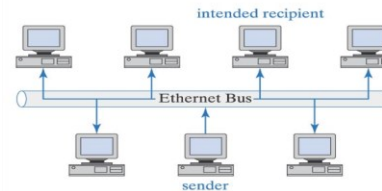


# Operating Systems and Networks

- ▶ subfield concerned with mechanisms to control the hardware and software components of computer systems
- ▶ application: *operating systems* mediate between hardware and software
  - ▶ *time-sharing* - allowed for multiple users to work on the same computer
    - ▶ each user is allocated a portion of the processor, and the processor rotates among tasks so rapidly that it appears to be executing tasks simultaneously
  - ▶ multitasking - a single user can run multiple programs simultaneously
    - ▶ each application is allocated a portion of the memory

application: *networks* allow computers to communicate and share resources

- wide area network (WAN) – connects computers over long distances (e.g., Internet)
- local area network (LAN) – connects computers over short distances (e.g., same room or building)
- Ethernet is a common, simple technology for building LANs



1. All computers connected to the Ethernet bus must constantly monitor its status.
2. If a computer wants to send a message, it first checks to make sure the bus is free (not currently transmitting a message).
3. If the bus is free, the sender labels the message to identify the desired recipient.
4. The labeled message is broadcast over the bus, reaching all connected computers.
5. Upon receipt, each computer checks the label to see if it is the intended recipient. If so, the message is read and processed. If not, the message is ignored.



# Software Engineering



subfield concerned with creating effective software systems

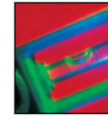
- large projects can encompass millions of lines of code
- teams of programmers work together to make an integrated whole
  - coordination and testing are key to successful projects

## Stages in the Software Life Cycle

1. *Requirement analysis and specification:* Initially, the needs of the customer must be assessed and the intended behavior of the software specified in detail.
2. *Design:* Next, the software system must be designed, including its breakdown into manageable pieces, and the underlying algorithms and data structures to be used.
3. *Implementation:* After completion of the design documents, the code must be written. For large software projects, teams may work independently on separate modules and then integrate those modules to complete the system.
4. *Testing:* The software must be systematically tested, both as independent modules and as a whole. As testing reveals errors or unforeseen conditions, reimplementation and even re-design may need to take place.
5. *Operation and maintenance:* Once the software system is complete, it must be installed and supported. It is estimated that maintenance, which involves fixing errors and updating code to meet changing customer needs, accounts for as much as half of a software project's total development budget.

- software demand continues to grow, placing pressure on programmers to produce at faster rates
  - clearly, there is a limit to personal productivity
  - simply adding more programmers does not solve the problem: increasing numbers means increased complexity, and coordination becomes an even bigger challenge
- in recent years, the adoption of the object-oriented programming methodology has made it easier to reuse code

# Artificial Intelligence



subfield that attempts to make computers exhibit human-like characteristics (e.g., the ability to reason and think)

- in 1950, Turing predicted intelligent computers by 2000 (still not even close)
- but, progress has been made in many A.I. realms
  - robots in manufacturing
  - expert systems – programs that encapsulate expert knowledge in a specific domain (e.g., for medical diagnosis)
  - neural computing – design of architectures that mimic the brain

## The Turing Test



In his 1950 paper, *Computing Machinery and Intelligence*, Alan Turing proposed what is still considered the ultimate test for artificial intelligence. The Turing Test is based on the observation that we, as humans, make assumptions about intelligence and self-awareness in other humans by monitoring and interacting with them. Turing proposed that if the behavior of a machine were indistinguishable from a human's behavior, then we ought to give the machine the same credit for being intelligent that we give people.

The Turing Test involves a human judge and two contestants, one being the computer to be tested and the second being a human control subject. The job of the judge is to converse with the two contestants via computer terminals, without knowing which contestant is which. If, after a sufficiently long period of conversation, the judge is unable to identify the computer, then the computer is said to have passed the test and must be considered to possess human-like intelligence.

# The Ethics of Computing



as technology becomes more prevalent in society, computing professionals must ensure that hardware and software are used safely, fairly, and effectively

## ACM Code of Ethics and Professional Conduct<sup>4</sup> Copyright © 1997, Association for Computing Machinery, Inc.

### **General Moral Imperatives.** *As an ACM member I will . . .*

- 1.1 Contribute to society and human well-being.
- 1.2 Avoid harm to others.
- 1.3 Be honest and trustworthy.
- 1.4 Be fair and take action not to discriminate.
- 1.5 Honor property rights including copyrights and patent.
- 1.6 Give proper credit for intellectual property.
- 1.7 Respect the privacy of others.
- 1.8 Honor confidentiality.

### **More Specific Professional Responsibilities.** *As an ACM computing professional I will . . .*

- 2.1 Strive to achieve the highest quality, effectiveness and dignity in both the process and products of professional work.
- 2.2 Acquire and maintain professional competence.
- 2.3 Know and respect existing laws pertaining to professional work.
- 2.4 Accept and provide appropriate professional review.
- 2.5 Give comprehensive and thorough evaluations of computer systems and their impacts, including analysis of possible risks.
- 2.6 Honor contracts, agreements, and assigned responsibilities.
- 2.7 Improve public understanding of computing and its consequences.
- 2.8 Access computing and communication resources only when authorized to do so.

# *What is Python...?*

- ❑ Python is a general purpose programming language that is often applied in scripting roles.
- ❑ So, Python is programming language as well as scripting language.
- ❑ Python is also called as Interpreted language
- ▶ Python is a popular high-level programming language used in various applications
  - ▶ Python is an easy language to learn because of its simple syntax
  - ▶ Python can be used for simple tasks such as plotting or for more complex tasks like machine learning



# *History*

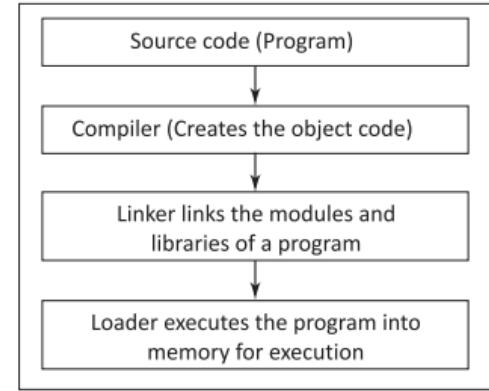
- ❑ Invented in the Netherlands, early 90s by Guido van Rossum
- ❑ Python was conceived in the late 1980s and its implementation was started in December 1989
- ❑ Guido Van Rossum is fan of 'Monty Python's Flying Circus', this is a famous TV show in Netherlands
- ❑ Named after Monty Python
- ❑ Open sourced from the beginning



# Overview of Programming Languages

- **Machine Language:** A computer is an electronic machine which can understand any instruction written in binary form, i.e. using only 0s and 1s. A program written in 0s and 1s is called machine language. While a computer easily understands this language, it is difficult for humans to write an instruction in terms of 0's and 1's. Consider the following example.
- **Assembly Language:** In assembly language, which was developed subsequently, machine operations are represented by mnemonic codes (such as ADD and MUL) and symbolic names that specify the memory address.

- **High Level Language:** High-level languages are much easier to write than low-level languages because programs written in these are similar to instructions written in the English language. Here 'high' does not imply that the language is complicated. It means that the language is more problem oriented. Generally, highlevel languages are platform independent. This means that one can write a program in a high-level language and run it on different types of machines. Instructions written in high-level languages are called statements.



**Figure 1.2** Steps to execute a high-level language program



**Compiler:** It is a software that translates a program written in a high-level language into machine language. This compiled program is called object code. The object code is an executable code which can run as a standalone code, i.e. it does not need the compiler to be present during execution. Every programming language, such as C, C++ and Java has its own compiler.

**Interpreter:** While a compiler converts the whole source code into an equivalent object code or machine code, the interpreter reads the source code line by line and converts it into object code (i.e. a code understandable to the machine).

**Linker:** It is a program that links different program modules and libraries to form a single executable program. A source code of a program is very large. It can consist of hundreds of lines of code. Before the execution of a program, all the modules of the program and the required libraries are linked together using a software called a linker. The compiled and linked program is called the executable code.

**Loader:** This software is used to load and relocate an executable program in the main memory during execution. The loader assigns a storage space to a program in the main memory for execution.

**Scripting Language:** Do not require compilation step (they are interpreted)



# *Differences between program and scripting language*

## **Program**

- a **program is executed** (*i.e. the source is first compiled, and the result of that compilation is expected*)
- A "program" in general, is a **sequence of instructions written so that a computer can perform certain task.**

## **Scripting**

- a **script is interpreted**
- A "script" is code written in a scripting language. A scripting language is nothing but a **type of programming language in which we can write code to control another software application.**



# Features of Python

Python stands out because of its simplicity and versatility, making it a top choice for both beginners and professionals. Here are some key features or characteristics:

- 1. Easy to Read and Write:** Python's syntax is clean and simple, making the code easy to understand and write, even for those new to programming.
- 2. Interpreted Language:** Python executes code line by line, which helps in easy debugging and testing during development.
- 3. Object-Oriented and Functional:** Python supports both object-oriented and functional programming, giving developers flexibility in how they structure their code.
- 4. Dynamically Typed:** You don't need to specify data types when declaring variables; Python figures it out automatically.
- 5. Extensive Libraries:** Python has a rich collection of libraries for tasks like web development, data analysis, machine learning, and more.
- 6. Cross-Platform:** Python can run on different operating systems like Windows, macOS, and Linux without modification.
- 7. Community Support:** Python has a large, active community that continuously contributes resources, libraries, and tools, making it easier to find help or solutions.

## *Why do people use Python...?*

The following primary factors cited by Python users seem to be these:

- **Python is object-oriented**

Structure supports such concepts as polymorphism, operation overloading, and multiple inheritance.

- **Indentation**

Indentation is one of the greatest feature in Python.

- **It's free (open source)**

Downloading and installing Python is free and easy  
Source code is easily accessible

## □ It's powerful

- Dynamic typing
- Built-in types and tools
- Library utilities
- Third party utilities (e.g. Numeric, NumPy, SciPy)
- Automatic memory management

## □ It's portable

- Python runs virtually every major platform used today
- As long as you have a compatible Python interpreter installed, Python programs will run in exactly the same manner, irrespective of platform.



### □ It's mixable

- Python can be linked to components written in other languages easily
- Linking to fast, compiled code is useful to computationally intensive problems
- - Python/C integration is quite common

### □ It's easy to use

- No intermediate compile and link steps as in C/ C++
- Python programs are compiled automatically to an intermediate form called *bytecode*, which the interpreter then reads
- This gives Python the development speed of an interpreter without the performance loss inherent in purely interpreted languages

### □ It's easy to learn

- Structure and syntax are pretty intuitive and easy to grasp



# Applications of Python

Python is widely used across various fields due to its flexibility and ease of use. Here are some of the main applications:

1. **Web Development:** Python, with frameworks like Django and Flask, is used to create dynamic websites and web applications quickly and efficiently.
2. **Data Science and Analytics:** Python is a go-to language for data analysis, visualization, and handling large datasets, thanks to libraries like Pandas, NumPy, and Matplotlib.
3. **Artificial Intelligence and Machine Learning:** Python is popular in AI and machine learning because of its powerful libraries like TensorFlow, Keras, and Scikit-learn.
4. **Automation:** Python is commonly used to automate repetitive tasks, making processes faster and more efficient.
5. **Game Development:** While not as common, Python is also used for game development, with libraries like Pygame to create simple games.
6. **Scripting:** Python's simplicity makes it ideal for writing scripts that automate tasks in different systems, from server management to file handling.
7. **Desktop GUI Applications:** Python can be used to build desktop applications using frameworks like Tkinter and PyQt.



# Python vs. Other Programming Languages

| Feature                      | Python                                                   | C                                                      | C++                                                                 | Java                                          |
|------------------------------|----------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------|
| Type                         | Interpreted                                              | Compiled                                               | Compiled                                                            | Compiled and Interpreted                      |
| Paradigm                     | Multi-paradigm (object-oriented, procedural, functional) | Procedural, structured                                 | Multi-paradigm (procedural, object-oriented, generic)               | Object-oriented, structured                   |
| Memory Management            | Automatic                                                | Manual                                                 | Manual                                                              | Automatic                                     |
| Syntax                       | Simple                                                   | Complex                                                | Complex                                                             | Complex                                       |
| Popularity                   | High                                                     | High                                                   | High                                                                | High                                          |
| Use Cases                    | Web development, data analysis, machine learning         | System programming, embedded systems, game development | System programming, game development, high-performance applications | Large-scale applications, enterprise software |
| Notable Frameworks/Libraries | Django, Flask                                            | Standard Library                                       | Standard Library, Boost                                             | Spring, Hibernate                             |
| Learning Curve               | Moderate                                                 | Steep                                                  | Steep                                                               | Moderate                                      |
| Community Support            | Strong                                                   | Strong                                                 | Strong                                                              | Strong                                        |
| Job Market                   | Abundant                                                 | Abundant                                               | Abundant                                                            | Abundant                                      |



# List of Companies Using Python

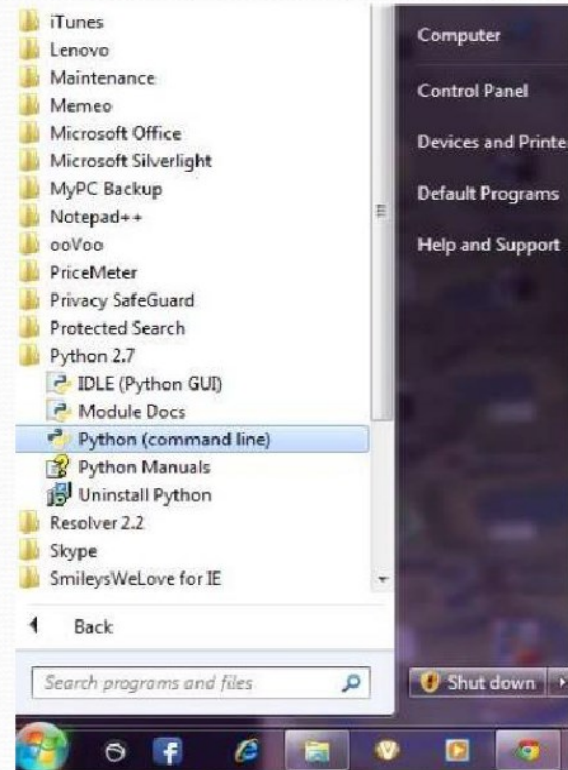
| Company   | Description                                                                                          |
|-----------|------------------------------------------------------------------------------------------------------|
| Google    | Uses Python for various applications, including their search engine and machine learning projects.   |
| Instagram | The backend of Instagram is built using Python, enabling it to handle millions of users efficiently. |
| Spotify   | Python is used for data analysis and backend services, helping improve user recommendations.         |
| Dropbox   | Python powers the desktop client of Dropbox, making it easy to sync files across devices.            |
| Netflix   | Python helps Netflix with data analysis and managing its content recommendation algorithms.          |
| Reddit    | One of the largest online communities, Reddit, uses Python for its core functionalities.             |
| Uber      | Uber uses Python for various features, including dynamic pricing and data analysis.                  |
| Pinterest | Python plays a key role in the backend of Pinterest, helping scale and manage user content.          |



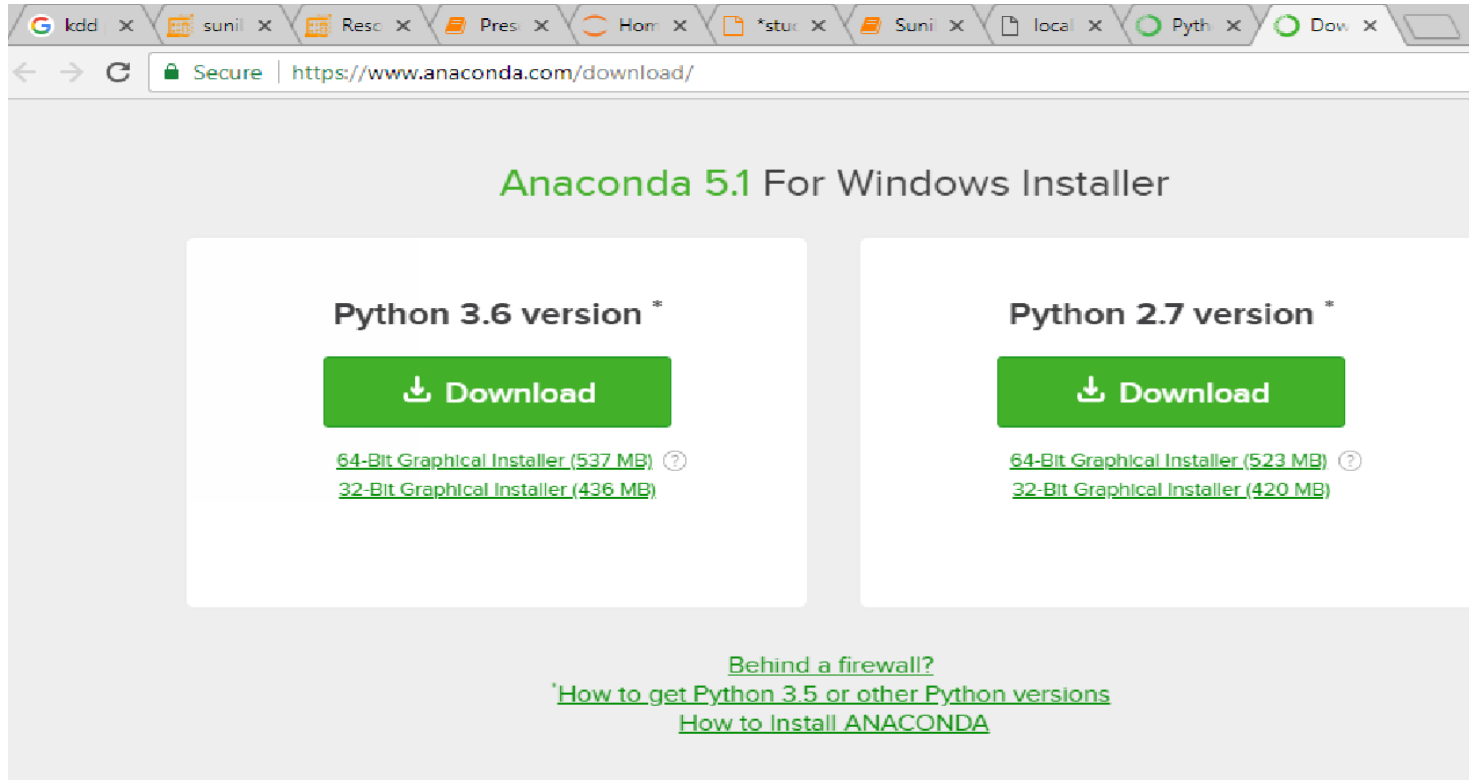
# *Installing Python*

- Python is pre-installed on most Unix systems, including Linux and MAC OS X
  
- But for in Windows Operating Systems , user can download from the <https://www.python.org/downloads/>
  - from the above link download latest version of python IDE and install, recent version is 3.4.1 but most of them uses version 2.7.7 only

- After installing the Python Ver#2.7.7, go to start menu then click on python 2.7 in that one you can select python (command line) it is prompt with >>>



# How to Install Anaconda



The screenshot shows a web browser window with multiple tabs. The active tab is displaying the Anaconda download page for Windows. The page title is "Anaconda 5.1 For Windows Installer". There are two main sections: "Python 3.6 version \*" and "Python 2.7 version \*". Each section has a green "Download" button with a download icon. Below the buttons, there are links for "64-Bit Graphical Installer" and "32-Bit Graphical Installer" with their respective sizes in MB. At the bottom, there are links for "Behind a firewall?", "How to get Python 3.5 or other Python versions", and "How to Install ANACONDA".

Python 3.6 version \*

[Download](#)

[64-Bit Graphical Installer \(537 MB\)](#) ?  
[32-Bit Graphical Installer \(436 MB\)](#)

Python 2.7 version \*

[Download](#)

[64-Bit Graphical Installer \(523 MB\)](#) ?  
[32-Bit Graphical Installer \(420 MB\)](#)

[Behind a firewall?](#)  
[How to get Python 3.5 or other Python versions](#)  
[How to Install ANACONDA](#)

# ANACONDA NAVIGATOR

[Home](#)[Environments](#)[Projects \(beta\)](#)[Learning](#)[Community](#)[Documentation](#)[Developer Blog](#)

Applications on

base (root)

Channels



jupyterlab

0.31.5

An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.

[Launch](#)

notebook

5.4.0

Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis.

[Launch](#)



Google

kdd

sunil

Resc

Pres

Hom

\*stuc

Suni

local

Pyth

Dow

localhost:8888/tree

☆

🔴

🔵

 jupyter

Logout

Files

Running

Clusters

Select items to perform actions on them.

0

📁 /

📁

Contacts

📁

Desktop

📁

Documents

📁

Downloads

📁

Favorites

6 months ago

📁

Links

5 months ago

📁

Music

3 months ago

📁

Pictures

2 months ago

📁

Saved Games

a month ago

📁

Searches

4 months ago

Upload

New

🔄

Notebook:

Python 3

Create a new notebook with Python

Text File

Folder

Terminal



Google Chrome browser window showing a Jupyter Notebook interface.

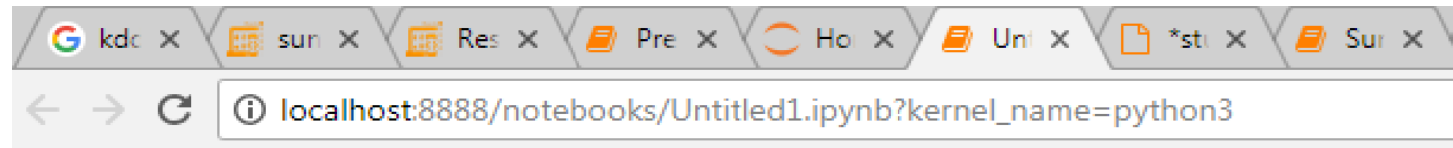
Address bar: `localhost:8888/notebooks/Untitled1.ipynb?kernel_name=python3`

Jupyter Notebook header: **Untitled1** Last Checkpoint: a few seconds ago (unsaved changes) Logout

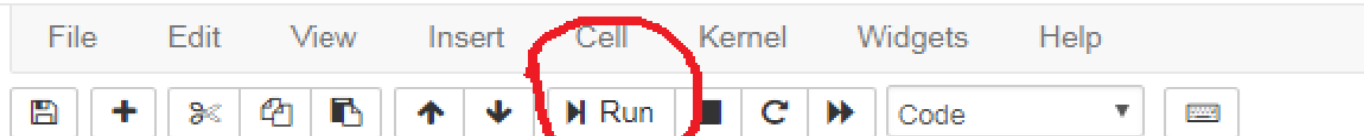
Menu bar: File Edit View Insert Cell Kernel Widgets Help

Toolbar: Save, Add, Find, Copy, Paste, Undo, Redo, Run, Stop, Restart, Code (dropdown), Help

Code cell input: In [ ]: |



jupyter Untitled1 Last Checkpoint: 2 minutes ago (autosaved)



```
In [1]: # To print Message use print function
        # To Execute Press Shift and Enter
        print("Hello World")
```

Hello World

In [ ]:

# Python in Different Execution Modes



- **Python (Command Line):** Python is an interpreted language. You can directly write the code into the Python interpreter or you can write a sequence of instructions into a file and then run the file. When you execute Python expressions or statements from the command line then you are in interactive mode or interactive prompt.

```
C:\Python34\python.exe
Python 3.4.2 (v3.4.2:ab2c023a9432, Oct 6 2014, 22:15:05) [MSC v.1600 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print('Hello World')
Hello World
>>> print('Bye for Now')
Bye for Now
>>> _
```





- **Python IDLE:** Launching Python IDLE is another way to start executing Python statements or commands in the interactive mode of Python IDLE. It is a graphical integrated development environment for Python. Python statements or commands which run in the interactive mode of Python IDLE are called shell. IDLE is downloaded by default while installing Python. Launching Python IDLE is the simplest way to open a Python shell. Running Python programs from a script file is known as running Python in script mode. You can write a sequence of instructions in one file and execute them.

```
Python 3.4.2 Shell
File Edit Shell Debug Options Windows Help
Python 3.4.2 (v3.4.2:ab2c023a9432, Oct 6 2014, 22:15:05) [MSC v.1600 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> print('Welcome Make Habit of It ')
Welcome Make Habit of It
>>> print(' Awesome Interactive Shell')
Awesome Interactive Shell
>>>
```

Ln: 2 Col: 0



# **IDLE is Python's Integrated Development and Learning Environment.**

IDLE has the following features:

- cross-platform: works mostly the same on Windows, Unix, and macOS
- Python shell window (interactive interpreter) with colorizing of code input, output, and error messages
- multi-window text editor with multiple undo, Python colorizing, smart indent, call tips, auto completion, and other features
- search within any window, replace within editor windows, and search through multiple files (grep)
- debugger with persistent breakpoints, stepping, and viewing of global and local namespaces
- configuration, browsers, and other dialogs



# Our First Program

- `print("Hello World", 7)`
- `print(5)`
- `print("Bye")`
- `print(17*13)`

## A Sample Code

```
x = 34 - 23      # A comment.  
y = "Hello"      # Another one.  
z = 3.45  
if z == 3.45 or y == "Hello":  
    x = x + 1  
    y = y + " World" # String concat.  
print x  
print y
```

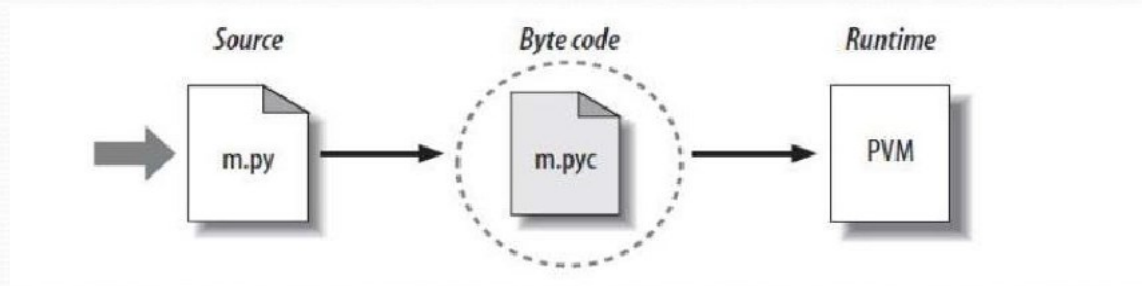
# Enough to understand the code

- Indentation matters to code meaning
  - Block structure indicated by indentation
- First assignment to a variable creates it
  - Variable types don't need to be declared.
  - Python figures out the variable types on its own.
- Assignment is `=` and comparison is `==`
- For numbers `+` `-` `*` `/` `%` are as expected
  - Special use of `+` for string concatenation and `%` for string formatting (as in C's `printf`)
- Logical operators are words (`and`, `or`, `not`) *not* symbols
- The basic printing command is `print`



# *Python Code Execution*

- Python's traditional runtime execution model: source code you type is translated to byte code, which is then run by the Python Virtual Machine. Your code is automatically compiled, but then it is interpreted.



Source code extension is **.py**

Byte code extension is **.pyc** (compiled **python** code)

# *Running Python*

Once you're inside the Python interpreter, type in commands at will.

- **Examples:**

```
>>> print 'Hello world'
```

```
Hello world
```

**# Relevant output is displayed on subsequent lines without the >>> symbol**

```
>>> x = [0,1,2]
```

**# Quantities stored in memory are not displayed by default**

```
>>> x
```

**# If a quantity is stored in memory, typing its name will display it**

```
[0,1,2]
```

```
>>> 2+3
```

```
5
```



# Comments, Escape sequence & Print in Python

## Python Comments

A comment is a part of the coding file that the programmer does not want to execute, rather the programmer uses it to either explain a block of code or to avoid the execution of a specific part of code while testing.

### Single-Line Comments:

To write a comment just add a '#' at the start of the line.

### Example 1

```
#This is a 'Single-Line Comment'  
print("This is a print statement.")
```

Output:

This is a print statement.

### Example 2

```
print("Hello World !!!") #Printing Hello World
```

Output:

Hello World !!!





## Multi-Line Comments:

To write multi-line comments you can use '#' at each line or you can use the multiline string.

**\*\*Example 1:\*\*** The use of '#'.  
#It will execute a block of code if a specified condition is true.  
#If the condition is false then it will execute another block of code.

```
p = 7
if (p > 5):
    print("p is greater than 5.")
else:
    print("p is not greater than 5.")
```

Output:  
p is greater than 5.

**\*\*Example 2:\*\*** The use of multiline string.

"""This is an if-else statement.

It will execute a block of code if a specified condition is true.

If the condition is false then it will execute another block of code."""



```
a=5  
print("The number is" , a)
```

## Escape Sequence Characters

To insert characters that cannot be directly used in a string, we use an escape sequence character.

An escape sequence character is a backslash `\\` followed by the character you want to insert.

An example of a character that cannot be directly used in a string is a double quote inside a string that is surrounded by double quotes:

```
print("This doesnt \"execute\")  
print("This will \\ \" execute")
```



## More on Print function

The syntax of a print statement looks something like this:

```
print(object(s), sep=separator, end=end, file=file, flush=flush)
```

### ## Other Parameters of Print Statement

1. object(s): Any object, and as many as you like. Will be converted to string before printed
2. sep='separator': Specify how to separate the objects, if there is more than one. Default is ' '
3. end='end': Specify what to print at the end. Default is '\n' (line feed)
4. file: An object with a write method. Default is sys.stdout

Parameters 2 to 4 are optional

# Keywords in Python

- Keywords in Python are reserved words that can not be used as a variable name, function name, or any other identifier.

|               |                |                 |
|---------------|----------------|-----------------|
| and           | False          | <u>nonlocal</u> |
| <u>as</u>     | <u>finally</u> | <u>not</u>      |
| <u>assert</u> | <u>for</u>     | or              |
| <u>break</u>  | from           | <u>pass</u>     |

|                 |               |               |
|-----------------|---------------|---------------|
| <u>class</u>    | <u>global</u> | <u>raise</u>  |
| <u>continue</u> | if            | <u>return</u> |
| <u>def</u>      | <u>import</u> | True          |
| <u>del</u>      | <u>is</u>     | <u>try</u>    |

|               |               |              |
|---------------|---------------|--------------|
| elif          | <u>in</u>     | <u>while</u> |
| else          | <u>lambda</u> | <u>with</u>  |
| <u>except</u> | <u>None</u>   | <u>yield</u> |



# Variables and Data Types

## What is a variable?

- Variable is like a container that holds data. Very similar to how our containers in kitchen holds sugar, salt etc
- Creating a variable is like creating a placeholder in memory and assigning it some value. In Python its as easy as writing:

`a = 1`

`b = True`

`c = "Harry"`

`d = None`

These are four variables of different data types.



# Rules for Python variables

- A Python variable name must start with a letter or the underscore character.
- A Python variable name cannot start with a number.
- A Python variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and \_ ).
- Variable in Python names are case-sensitive (name, Name, and NAME are three different variables).
- The reserved words(keywords) in Python cannot be used to name the variable in Python.



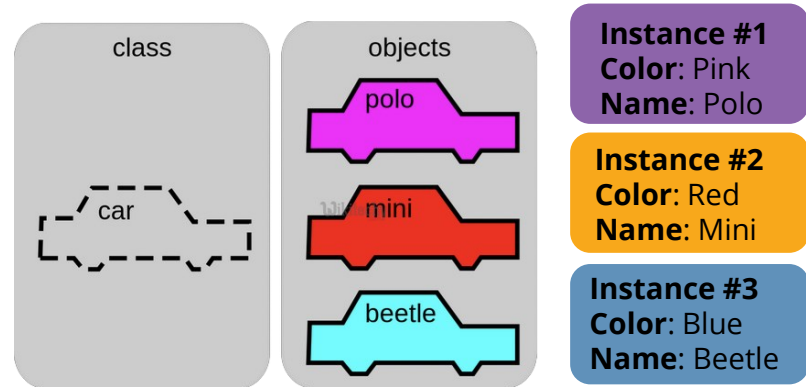
# Variables, Objects, and Classes

- A **variable** is a reference to a value stored in a computer's memory.
- Variables can be sorted into a variety of categories (or **data types**) such as **numbers (int/float etc)**, **Boolean values (true/false)**, and **sequences (strings, lists etc)**.
- An **object** is a collection of data from a computer's memory that can be manipulated.
  - **ALL VARIABLES ARE OBJECTS** although some objects can be defined by data referred to by multiple variables.
  - **Methods** are the functions used to act on/alter an object's data. They describe what your object can "do."



# Variables, Objects, and Classes (cont.)

- A **class** is a collection of objects who share the same set of variables/methods.
  - The definition of the class provides a blueprint for all the objects within it (**instances**).
  - Instances may share the same variables (color, size, shape, etc.), but they do **NOT** share the same values for each variable (blue/red/pink, small/large, square/circular etc.)





# Basic Syntax Rules

- The name of your variable (**myInt** etc.) is placed on the left of the "=" operator.
  - Most variable names are in **camel case** where the first word begins with a lowercase letter and any subsequent words are capitalized
  - Variable names may also appear in **snake case** where all words are lowercase, with underscores between words
- The assignment operator ("=") sets the variable name equal to the memory location where your value is found.
- The value of your variable ("**Hello, World**") is placed on the right of the "=" operator.
  - The type of this value does **NOT** need to be stated but its format must abide by a given object type (as shown).

```
myString = "Hello, World" myInt = 7  
myFloat = 7.0  
myList = [7, 8, 9] myBoolean = true
```



# What is a Data Type?

Data type specifies the type of value a variable holds. This is required in programming to do various operations without causing an error.

*Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data. Since everything is an object in Python programming, data types are classes and variables are instances (objects) of these classes.*

In python, we can print the type of any operator using type function:

```
a = 1  
print(type(a))
```

```
b = "1"  
print(type(b))
```

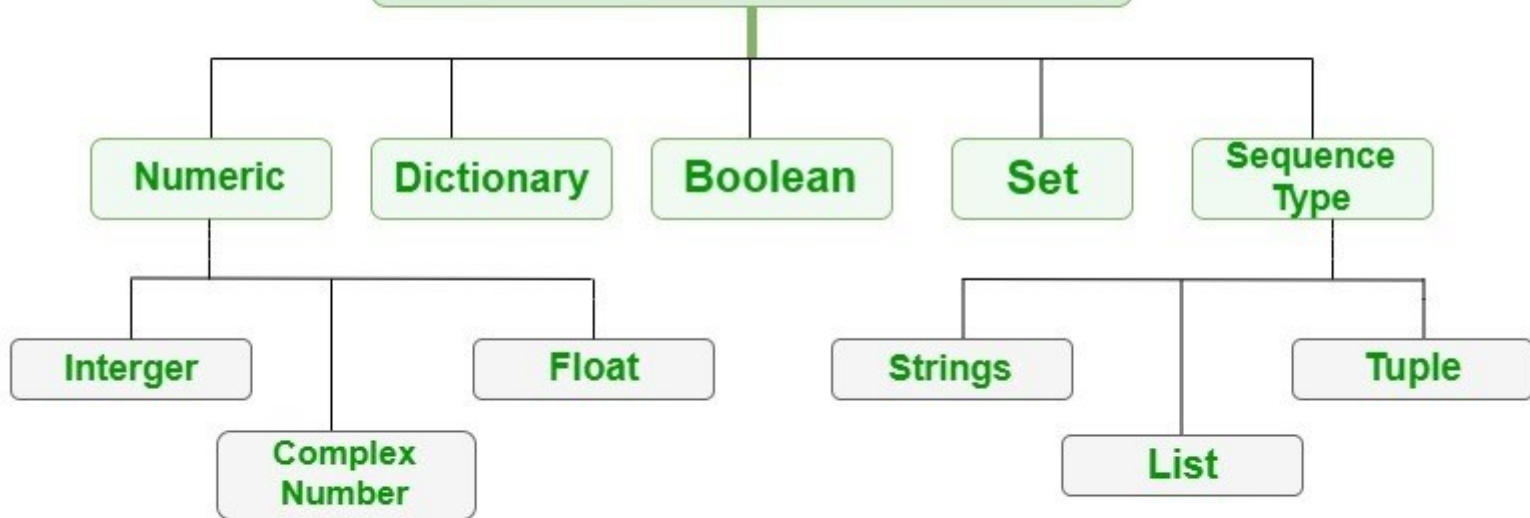
# Common Data Types and Operators

- A **data type** is a means of classifying a value and determining what operations can be performed on it. All objects have a data type.
- **Operators** are symbols used carry out specific functions/computations.
- <https://www.youtube.com/watch?v=v5MR5JnKcZI>

| Operator          | Description                                                                     |
|-------------------|---------------------------------------------------------------------------------|
| **                | Exponentiation (raise to the power)                                             |
| ~ + -             | Ccomplement, unary plus and minus (method names for the last two are +@ and -@) |
| * / % //          | Multiply, divide, modulo and floor division                                     |
| + -               | Addition and subtraction                                                        |
| >> <<             | Right and left bitwise shift                                                    |
| &                 | Bitwise 'AND'                                                                   |
| ^                 | Bitwise exclusive 'OR' and regular 'OR'                                         |
| <= < > >=         | Comparison operators                                                            |
| <> == !=          | Equality operators                                                              |
| = %= /= //= -= += | Assignment operators                                                            |
| *= **=            |                                                                                 |
| is is not         | Identity operators                                                              |
| in not in         | Membership operators                                                            |
| not or and        | Logical operators                                                               |

| Class     | Description                          |
|-----------|--------------------------------------|
| bool      | Boolean value                        |
| int       | integer (arbitrary magnitude)        |
| float     | floating-point number                |
| list      | mutable sequence of objects          |
| tuple     | immutable sequence of objects        |
| str       | character string                     |
| set       | unordered set of distinct objects    |
| frozenset | immutable form of set class          |
| dict      | associative mapping (aka dictionary) |

## Python - Data Types





By default, python provides the following built-in data types:

### 1. Numeric data: int, float, complex

- int: 3, -8, 0
- float: 7.349, -9.0, 0.0000001
- complex: 6 + 2i

### 2. Text data: str

str: "Hello World!!!", "Python Programming"

### 3. Boolean data:

Boolean data consists of values True or False.

### 4. Sequenced data: list, tuple:

**List:** A list is an ordered collection of data with elements separated by a comma and enclosed within square brackets. Lists are mutable and can be modified after creation.

Example:

```
list1 = [8, 2.3, [-4, 5], ["apple", "banana"]]  
print(list1)
```



**Tuple:** A tuple is an ordered collection of data with elements separated by a comma and enclosed within parentheses. Tuples are immutable and can not be modified after creation.

Example:

```
tuple1 = (("parrot", "sparrow"), ("Lion", "Tiger"))  
print(tuple1)
```

## 5. Mapped data: dict

**Dict:** A dictionary is an unordered collection of data containing a key:value pair. The key:value pairs are enclosed within curly brackets.

Example:

```
dict1 = {"name":"Sakshi", "age":20, "canVote":True}  
print(dict1)
```

Output:

```
{'name': 'Sakshi', 'age': 20, 'canVote': True}
```



# Python Operators

- In Python programming, Operators in general are used to perform operations on values and variables. These are standard symbols used for the purpose of logical and arithmetic operations. In this article, we will look into different types of Python operators.
- Arithmetic Operators
- Python Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication, and division.





```
a = 9
b = 4
add = a + b
sub = a - b
mul = a * b
mod = a % b
p = a ** b
print(add)
print(sub)
print(mul)
print(mod)
print(p)
```

Output:

13

5

36

1

6561

|                                         | Operator | Description                                                                    | Syntax   |
|-----------------------------------------|----------|--------------------------------------------------------------------------------|----------|
| <a href="#">Addition Operator</a>       | +        | Addition: adds two operands                                                    | $x + y$  |
| <a href="#">Subtraction Operator</a>    | -        | Subtraction: subtracts two operands                                            | $x - y$  |
| <a href="#">Multiplication Operator</a> | *        | Multiplication: multiplies two operands                                        | $x * y$  |
| <a href="#">Division Operator</a>       | /        | Division (float): divides the first operand by the second                      | $x / y$  |
| <a href="#">Floor Division Operator</a> | //       | Division (floor): divides the first operand by the second                      | $x // y$ |
| <a href="#">Modulus Operator</a>        | %        | Modulus: returns the remainder when the first operand is divided by the second | $x \% y$ |
| <a href="#">Exponentiation Operator</a> | **       | Power (Exponent): Returns first raised to power second                         | $x ** y$ |



# Logical Operators

Python Logical operators perform Logical AND , Logical OR , and Logical NOT operations. It is used to combine conditional statements.

```
a = True  
b = False  
print(a and b)  
print(a or b)  
print(not a)
```

**Output:**  
False  
True  
False



# Python Bitwise Operators

Python bitwise operators are used to perform bitwise calculations on integers. The integers are first converted into binary and then operations are performed on each bit or corresponding pair of bits, hence the name bitwise operators. The result is then returned in decimal format.

| OPERATOR                                   | NAME | DESCRIPTION         | SYNTAX                                                                                               |              |
|--------------------------------------------|------|---------------------|------------------------------------------------------------------------------------------------------|--------------|
| <a href="#">Bitwise AND operator</a>       | &    | Bitwise AND         | Result bit 1, if both operand bits are 1; otherwise results bit 0.                                   | $x \& y$     |
| <a href="#">Bitwise OR operator</a>        |      | Bitwise OR          | Result bit 1, if any of the operand bit is 1; otherwise results bit 0.                               | $x   y$      |
| <a href="#">Bitwise XOR Operator</a>       | ^    | Bitwise XOR         | Result bit 1, if any of the operand bit is 1 but not both, otherwise results bit 0.                  | $x \wedge y$ |
| <a href="#">Bitwise NOT Operator</a>       | ~    | Bitwise NOT         | Inverts individual bits.                                                                             | $\sim x$     |
| <a href="#">Python Bitwise Right Shift</a> | >>   | Bitwise right shift | The left operand's value is moved toward right by the number of bits specified by the right operand. | $x >>$       |
| <a href="#">Python Bitwise Left Shift</a>  | <<   | Bitwise left shift  | The left operand's value is moved toward left by the number of bits specified by the right operand.  | $x <<$       |



# Assignment Operators

Python Assignment operators are used to assign values to the variables.

```
a = 10  
b = a  
print(b)
```

```
b += a  
print(b)
```

```
b -= a  
print(b)
```

```
b *= a  
print(b)
```

**Output:**

10  
20  
10  
100

| Operators                                                 | Sign | Description                                                                                       | Syntax    |
|-----------------------------------------------------------|------|---------------------------------------------------------------------------------------------------|-----------|
| <a href="#"><u>Assignment Operator</u></a>                | =    | Assign the value of the right side of the expression to the left side operand                     | c = a + b |
| <a href="#"><u>Addition Assignment Operator</u></a>       | +=   | Add right side operand with left side operand and then assign the result to left operand          | a += b    |
| <a href="#"><u>Subtraction Assignment Operator</u></a>    | -=   | Subtract right side operand from left side operand and then assign the result to left operand     | a -= b    |
| <a href="#"><u>Multiplication Assignment Operator</u></a> | *=   | Multiply right operand with left operand and then assign the result to the left operand           | a *= b    |
| <a href="#"><u>Division Assignment Operator</u></a>       | /=   | Divide left operand with right operand and then assign the result to the left operand             | a /= b    |
| <a href="#"><u>Modulus Assignment Operator</u></a>        | %=   | Divides the left operand with the right operand and then assign the remainder to the left operand | a %= b    |

| Operators                                               | Sign | Description                                                                                     | Syntax   |
|---------------------------------------------------------|------|-------------------------------------------------------------------------------------------------|----------|
| <a href="#">Floor Division Assignment Operator</a>      | //=  | Divide left operand with right operand and then assign the value(floor) to left operand         | a //= b  |
| <a href="#">Exponentiation Assignment Operator</a>      | **=  | Calculate exponent(raise power) value using operands and then assign the result to left operand | a **= b  |
| <a href="#">Bitwise AND Assignment Operator</a>         | &=   | Performs Bitwise AND on operands and assign the result to left operand                          | a &= b   |
| <a href="#">Bitwise OR Assignment Operator</a>          | =    | Performs Bitwise OR on operands and assign the value to left operand                            | a  = b   |
| <a href="#">Bitwise XOR Assignment Operator</a>         | ^=   | Performs Bitwise XOR on operands and assign the value to left operand                           | a ^= b   |
| <a href="#">Bitwise Right Shift Assignment Operator</a> | >>=  | Performs Bitwise right shift on operands and assign the result to left operand                  | a >>= b  |
| <a href="#">Bitwise Left Shift Assignment Operator</a>  | <<=  | Performs Bitwise left shift on operands and assign the result to left operand                   | a <<= b  |
| <a href="#">Walrus Operator</a>                         | :=   | Assign a value to a variable within an expression                                               | a := exp |

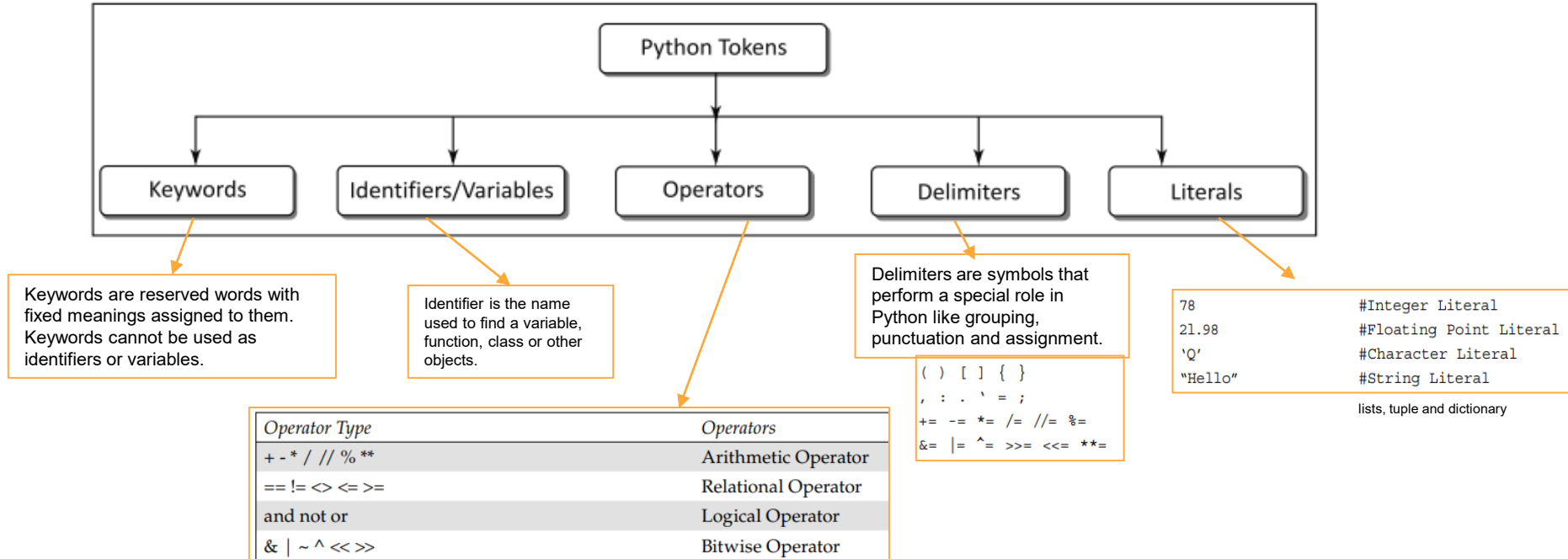
# Relational/Conditional Operators

| Operator                          | Meaning                  |
|-----------------------------------|--------------------------|
| <code>==</code> (double equal to) | Equal to                 |
| <code>&lt;</code>                 | Less than                |
| <code>&gt;</code>                 | Greater than             |
| <code>!=</code>                   | Not equal to             |
| <code>&lt;=</code>                | Less than or equal to    |
| <code>&gt;=</code>                | Greater than or equal to |



# Token

A program in Python contains a sequence of instructions. Python breaks each statement into a sequence of lexical components known as **tokens**. Each token corresponds to a substring of a statement. Python contains various types of tokens. Figure 2.1 shows the list of tokens supported by Python.





# Typecasting in python

The conversion of one data type into the other data type is known as type casting in python or type conversion in python.

Python supports a wide variety of functions or methods like: `int()`, `float()`, `str()`, `ord()`, `hex()`, `oct()`, `tuple()`, `set()`, `list()`, `dict()`, etc. for the type casting in python.

- **Two Types of Typecasting:**

1. Explicit Conversion (Explicit type casting in python)
2. Implicit Conversion (Implicit type casting in python).



# Explicit typecasting:

- The conversion of one data type into another data type, done via developer or programmer's intervention or manually as per the requirement, is known as explicit type conversion.

It can be achieved with the help of Python's built-in type conversion functions such as `int()`, `float()`, `hex()`, `oct()`, `str()`, etc.

- Example:

```
string = "15"
```

```
number = 7
```

```
string_number = int(string) #throws an error if the string is not a valid integer
```

```
sum= number + string_number
```

```
print("The Sum of both the numbers is: ", sum)
```

Output:

```
The Sum of both the numbers is 22
```



# Implicit type casting:

- Data types in Python do not have the same level i.e. ordering of data types is not the same in Python. Some of the data types have higher-order, and some have lower order. While performing any operations on variables with different data types in Python, one of the variable's data types will be changed to the higher data type. According to the level, one data type is converted into other by the Python interpreter itself (automatically). This is called, implicit typecasting in python.
- Python converts a smaller data type to a higher data type to prevent data loss.



# Example of Implicit type casting

# Python automatically converts

# a to int

a = 7

print(type(a))

b = 3.0

print(type(b))

# Python automatically converts c to float as it is a float addition

c = a + b

print(c)

print(type(c))

```
<class 'int'>
<class 'float'>
10.0
<class 'float'>
```



# Taking User Input in python

- In python, we can take user input directly by using input() function. This input function gives a return value as string/character hence we have to pass that into a variable
- **Syntax: variable=input()**
- But input function returns the value as string. Hence we have to typecast them whenever required to another datatype.
- Example:  

```
variable=int(input())  
variable=float(input())
```
- We can also display a text using input function. This will make input() function take user input and display a message as well.
- Example :  

```
a=input("Enter the name: ")  
print(a)
```



# What are strings?

- In python, anything that you enclose between single or double quotation marks is considered a string. A string is essentially a sequence or array of textual data. Strings are used when working with Unicode characters.

```
name = "Prachi"
```

```
print("Hello, " + name)
```

#Output: Hello, Prachi

- **Multiline Strings**

If our string has multiple lines, we can create them like this:

```
a = """Lorem ipsum dolor sit amet,  
consectetur adipiscing elit,  
sed do eiusmod tempor incididunt  
ut labore et dolore magna aliqua."""  
print(a)
```



# Chapter-2





# DECISION MAKING STATEMENTS

Sometimes the programmer needs to check the evaluation of certain expression(s), whether the expression(s) evaluate to True or False. If the expression evaluates to False, then the program execution follows a different path than it would have if the expression had evaluated to True.

Based on this, the conditional statements are further classified into following types:

- if
- if-else
- if-else-elif
- nested if-else-elif.



# An if.....else statement evaluates like this:

## **if the expression evaluates True:**

Execute the block of code inside if statement. After execution return to the code out of the if.....else block.\

## **if the expression evaluates False:**

Execute the block of code inside else statement. After execution return to the code out of the if.....else block.

### **Example:**

```
applePrice = 210
```

```
budget = 200
```

```
if (applePrice <= budget):
```

```
    print("Alexa, add 1 kg Apples to the cart.")
```

```
else:
```

```
    print("Alexa, do not add Apples to the cart.")
```

**Output:**

Alexa, do not add Apples to the cart.

# Example

```
a = int(input("Enter your age: "))
print("Your age is:", a)
# Conditional operators
# >, <, >=, <=, ==, !=
# print(a>18)
# print(a<=18)
# print(a==18)
# print(a!=18)
if(a>18):
    print("You can drive")
    print("Yes")
else:
    print("You cannot drive")
    print("No")
```

# elif Statements



Sometimes, the programmer may want to evaluate more than one condition, this can be done using an elif statement.

## Working of an elif statement

Execute the block of code inside if statement if the initial expression evaluates to True. After execution return to the code out of the if block.

Execute the block of code inside the first elif statement if the expression inside it evaluates True. After execution return to the code out of the if block.

Execute the block of code inside the second elif statement if the expression inside it evaluates True. After execution return to the code out of the if block.

.

Execute the block of code inside the nth elif statement if the expression inside it evaluates True. After execution return to the code out of the if block.

Execute the block of code inside else statement if none of the expression evaluates to True. After execution return to the code out of the if block.

```
num = 0
if (num < 0):
    print("Number is negative.")
elif (num == 0):
    print("Number is Zero.")
else:
    print("Number is positive.")
```

Output:  
Number is Zero.



# Nested if statements

We can use if, if-else, elif statements inside other if statements as well.

Example:

```
num = 18
if (num < 0):
    print("Number is negative.")
elif (num > 0):
    if (num <= 10):
        print("Number is between 1-10")
    elif (num > 10 and num <= 20):
        print("Number is between 11-20")
    else:
        print("Number is greater than 20")
else:
    print("Number is zero")
```

Output:  
Number is between 11-20



# Exercise 1

Write a program to test whether a number is divisible by 5 and 10 or by 5 or 10

```
num = int(input('Enter the number:'))  
print ('Entered Number is: ',num)  
  
if (num % 5 == 0 and num % 10==0):  
    print(num,' is divisible by both 5 and 10')  
if (num % 5 == 0 or num % 10 == 0):  
    print (num,'is divisible by 5 or 10')  
else:  
    print (num,' is not divisible either by 5 or 10')
```



# Exercise 2

Write a program that prompts a user to enter two different numbers. Perform basic arithmetic operations based on the choices.

```
num1 = float(input("Enter the first number:"))
num2=float(input("Enter the Second number:"))
print ("1) Addition ")
print ("2) Subtraction ")
print ("3) Multiplication ")
print ("4) Division ")
choice = int (input("Please Enter the Choice:"))
if choice==1:
    print (" Addition of ",num1,"and",num2,"is:",num1+num2)
elif choice==2:
    print (" Subtraction of ",num1,"and",num2,"is:",num1-num2)
elif choice==3:
    print (" Multiplication of ",num1,"and",num2,"is:",num1*num2)
elif choice==4:
    print (" Division of ",num1,"and",num2,"is:",num1/num2)
else:
    print ("Sorry!!! Invalid Choice")
```

# CONDITIONAL EXPRESSIONS



Consider the following piece of code.

```
if x%2==0:
```

```
    x = x*x
```

```
else:
```

```
    x = x*x*x
```

In the above code, initially, x is divided by 2. If x is divisible by 2 then the square of the number is assigned to variable x, else the cube of the number is assigned. To improve the performance of simple if-else statements, Python provides a conditional expression. Using this conditional expression, the code above can be rewritten as:

```
x=x*x if x % 2 == 0 else x*x*x
```

Therefore, the general form of conditional expression is:

**Expression<sub>1</sub> if condition else Expression<sub>2</sub>**

Expression<sub>1</sub> is the value of the conditional expression if the condition is true. Condition is a normal Boolean expression that generally appears in front of an if statement. Expression<sub>2</sub> is the value of the conditional expression if the condition is false.





# Exercise 3

Write a program to find the smaller number among the two numbers.

```
num1=int(input('Enter First Numbers:'))
num2=int(input('Enter Second Numbers:'))
if num1 < num2:
    min=num1
    print('min = ',min)
else:
    min=num2
    print('min = ',min)
```

The same program can be written using conditional expression as follows:

```
num1=int(input('Enter two Numbers:'))
num2=int(input('Enter two Numbers:'))
min = print('min = ',num1) if num1 < num2 else print('min = ',num2)
```

## Exercise 4

- Write a program to prompt a user to read the marks of five different subjects. Calculate the total marks and percentage of the marks and display the message according to the range of percentage given in table.

| Percentage                  | Message      |
|-----------------------------|--------------|
| per $\geq 90$               | Distinction  |
| per $\geq 80$ && per $< 90$ | First Class  |
| per $\geq 70$ && per $< 80$ | Second Class |
| per $\geq 60$ && per $< 70$ | First Class  |
| per $< 60$                  | Fail         |



```
Subject1=float(input("Enter the Marks of Data-Structure:"))
Subject2=float(input("Enter the Marks of Python:"))
Subject3=float(input("Enter the Marks of Java:"))
Subject4=float(input("Enter the Marks of C Programming:"))
Subject5=float(input("Enter the Marks of HTML:"))
sum=Subject1+Subject1+Subject3+Subject4+Subject5
per=sum/5
print("Total Marks Obtained", sum, "Out of 500")
print("Percentage = ",per)
if per>=90:
    print("Distinction")
else:
    if per>=80:
        print(" First Class")
    else:
        if per>=70:
            print("Second Class")
        else:
            if per>=60:
                print("Pass")
            else:
                print("Fail")
```

```
if per>=90:  
    print("Distinction")  
else:  
    if per>=80:  
        print(" First Class")  
    else:  
        if per>=70:  
            print("Second Class")  
        else:  
            if per>=60:  
                print("Pass")  
            else:  
                print("Fail")
```

Equivalent

```
if per>=90:  
    print("Distinction")  
elif per>=80:  
    print(" First Class")  
elif per>=70:  
    print("Second Class")  
elif per>=60:  
    print("Pass")  
else:  
    print("Fail")
```



# Example:

```
import time
timestamp=time.strftime("%H:%M:%S")
hourstamp=time.strftime("%H")
print(hourstamp)
if(hourstamp >= 0 and hourstamp <= 11):
    print("Good Morning!")
elif(hourstamp >= 12 and hourstamp <= 17):
    print("Good Afternoon")
else:
    print("Good Evening")
```



# Match Case Statements

To implement switch-case like characteristics very similar to if-else functionality, we use a match case in python. It is similar to switch-case statements of C, C++ or Java language.

A match statement will compare a given variable's value to different shapes, also referred to as the pattern. The main idea is to keep on comparing the variable with all the present patterns until it fits into one.

The match case consists of three main entities :

1. The match keyword
2. One or more case clauses
3. Expression for each case

The case clause consists of a pattern to be matched to the variable, a condition to be evaluated if the pattern matches, and a set of statements to be executed if the pattern matches.

**Syntax:**

```
match variable_name:
    case 'pattern1' : //statement1
    case 'pattern2' : //statement2
    ...
    case 'pattern n' : //statement n
```



## Example:

```
x = 4
# x is the variable to match
match x:
    # if x is 0
    case 0:
        print("x is zero")
    # case with if-condition
    case 4 if x % 2 == 0:
        print("x % 2 == 0 and case is 4")
    # Empty case with if-condition
    case _ if x < 10:
        print("x is < 10")
    # default case(will only be matched if the above cases were not matched)
    # so it is basically just an else:
    case _:
        print(x)
```

Output:

x % 2 == 0 and case is 4



# Introduction to Loops

Sometimes a programmer wants to execute a group of statements a certain number of times. This can be done using loops. Based on this loops are further classified into following main types;

- for loop
- while loop

## The for Loop

For loops can iterate over a sequence of iterable objects in python. Iterating over a sequence is nothing but iterating over strings, lists, tuples, sets and dictionaries.

### Example: iterating over a string:

```
name = 'Abhishek'
for i in name:
    print (i, end=" ", " ")
```

Output:  
A, b, h, i, s, h, e, k,

### Example: iterating over a list:

```
colors = ["Red", "Green", "Blue", "Yellow"]
for x in colors:
    print(x)
```

Output:  
Red  
Green  
Blue  
Yellow





```
name = 'Abhishek'
for i in name:
    print(i)
    if(i == "s"):
        print("S also stands for SATI!")
```

```
colors = ["Red", "Green", "Blue", "Yellow"]
for color in colors:
    print(color)
    for i in color:
        print(i)
```



# range():

What if we do not want to iterate over a sequence? What if we want to use for loop for a specific number of times?

Here, we can use the range() function.

## Example:

```
for k in range(5):  
    print(k)
```

Here, we can see that the loop starts from 0 by default and increments at each iteration.

But we can also loop over a specific range.

## Example:

```
for k in range(4,9):  
    print(k)
```

There is third parameter of range also: range(x, y, z)

Output:

0  
1  
2  
3  
4

Output:

4  
5  
6  
7  
8



# Definition of range ()

There is a inbuilt function in Python called **range()**, which is used to generate a list of integers. The range function has one, two or three parameters. The last two parameters in **range()** are optional.

The general form of the range function is:

**range(begin, end, step)**

The 'begin' is the first beginning number in the sequence at which the list starts.

The 'end' is the limit, i.e. the last number in the sequence.

The 'step' is the difference between each number in the sequence.

| <i>Example of Range Function</i> | <i>Output</i>                |
|----------------------------------|------------------------------|
| range(5)                         | [0, 1, 2, 3, 4]              |
| range(1,5)                       | [1, 2, 3, 4]                 |
| range(1,10,2)                    | [1, 3, 5, 7, 9]              |
| range(5,0,-1)                    | [5, 4, 3, 2, 1]              |
| range(5,0,-2)                    | [5, 3, 1]                    |
| range (-4,4)                     | [-4, -3, -2, -1, 0, 1, 2, 3] |
| range (-4,4,2)                   | [-4, -2, 0, 2]               |
| range(0,1)                       | [0]                          |
| range(1,1)                       | Empty                        |
| range(0)                         | Empty                        |



# While Loop

As the name suggests, while loops execute statements while the condition is True. As soon as the condition becomes False, the interpreter comes out of the while loop.

## Example:

```
count = 5
while (count > 0):
    print(count)
    count = count - 1
```

Output:

5  
4  
3  
2  
1

Here, the count variable is set to 5 which decrements after each iteration. Depending upon the while loop condition, we need to either increment or decrement the counter variable (the variable count, in our case) or the loop will continue forever.



```
for i in range (5):  
    print (i)
```

```
i=0  
while (i<5):  
    print(i)  
    i=i+1  
print("We are out of loop")
```



# Else with While Loop

We can even use the else statement with the while loop. Essentially what the else statement does is that as soon as the while loop condition becomes False, the interpreter comes out of the while loop and the else statement is executed.

## Example:

```
x = 5
while (x > 0):
    print(x)
    x = x - 1
else:
    print('counter is 0')
```

Output:

5

4

3

2

1

counter is 0

# If-else Statements

- **If-else statements** allow programmers to adapt the function of their code based on a given condition.
- If a given condition (i.e. `x % 2 == 0`) is true, then the statements following the if statement (**if**) will be executed. If the condition is false, the statements following the else statement (**else**) will be executed.
  - The condition is tested using the Boolean operators `==` (is equal to), `!=` (is not equal to), **and** (used to test multiple conditions), and **or** (used to test if **AT LEAST ONE** condition is true).
  - Additionally, **else-if statements** (**elif**) can be used to provide unique coding statements for multiple conditions.

```
xString = input("Enter a number: ")
x = int(xString)
if x % 2 == 0:
    print("This is an even number")
elif x == 0:
    print("This number equals 0")
else:
    print("This is an odd number")
```





# For Loops

- **For loops** perform the same task (iterate) for the number of times specified by an **iterable** (something that can be evaluated repeatedly such as a list, string, or range).
- **for** defines the for loop
- **x** is the variable defining the number of times the statements within the loop (**print(myInt)**) are executed.
- The **range(start, stop, step)** function is often used to define x.
  - The starting value is defined by **start**, the final value is defined by **stop - 1**, and the magnitude at which x changes between loops is defined by **step**.
- **in** is a Boolean operator that returns true if the given value (x) is found within a given list, string, range etc.

```
myString = input("Enter a number: ")  
myInt = int(myString)
```

```
for x in range(0, 5, 1): print(myInt)
```



# While Loops

- **While loops** are statements that iterate so long as a given Boolean condition is met.
  - **x** (the variable determining whether or not the condition is met) is defined and manipulated **OUTSIDE** of the header of the while loop (**while**)
  - The condition (**x < 5**) is a statement containing a Boolean variable.
  - **break** is a statement used to exit the current for/while loop.
  - **continue** is a statement used to reject all statements in the current for/while loop iteration and return to the beginning of the loop.

```
myString = input("Enter a number: ")
myInt = int(myString)
```

```
x = 0
```

```
while x < 5:
```

```
    print(myInt)
```

```
    x = x + 1
```

```
1  # Prints out 0,1,2,3,4
2
3  count = 0
4  while True:
5      print(count)
6      count += 1
7      if count >= 5:
8          break
9
10 # Prints out only odd numbers - 1,3,5,7,9
11 for x in range(10):
12     # Check if x is even
13     if x % 2 == 0:
14         continue
15     print(x)
```

# Do-While loop in python



do..while is a loop in which a set of instructions will execute at least once (irrespective of the condition) and then the repetition of loop's body will depend on the condition passed at the end of the while loop. It is also known as an exit-controlled loop.

## How to emulate do while loop in python?

To create a do while loop in Python, you need to modify the while loop a bit in order to get similar behavior to a do while loop.

The most common technique to emulate a do-while loop in Python is to use an infinite while loop with a break statement wrapped in an if statement that checks a given condition and breaks the iteration if that condition becomes true:

### Example:

```
while True:
    number = int(input("Enter a positive number: "))
    print(number)
    if not number > 0:
        break
```

### Output

```
Enter a positive number: 1
1
Enter a positive number: 4
4
Enter a positive number: -1
-1
```

This loop uses True as its formal condition. This trick turns the loop into an infinite loop. Before the conditional statement, the loop runs all the required processing and updates the breaking condition. If this condition evaluates to true, then the break statement breaks out of the loop, and the program execution continues its normal path.



# break statement

The break statement enables a program to skip over a part of the code. A break statement terminates the very loop it lies within.

## Example:

```
for i in range(1,101,1):  
    print(i ,end=" ")  
    if(i==50):  
        break  
    else:  
        print("Mississippi")  
print("Thank you")
```

## Output:

```
1 Mississippi  
2 Mississippi  
3 Mississippi  
4 Mississippi  
5 Mississippi  
.  
.  
.  
50 Mississippi
```



```
print ("Multiplication Table of 5")
for i in range (1, 25):
    print("5 x",i, "=", 5*i)
    if (i==10):
        break
```



# Continue Statement

The continue statement skips the rest of the loop statements and causes the next iteration to occur.

**Example:**

```
for i in [2,3,4,6,8,0]:  
    if (i%2!=0):  
        continue  
    print(i)
```

**output**

2  
4  
6  
8  
0



# Chapter-3



# What are strings?

- In python, anything that you enclose between single or double quotation marks is considered a string. A string is essentially a sequence or array of textual data. Strings are used when working with Unicode characters.

```
name = "Prachi"
```

```
print("Hello, " + name)
```

```
#Output: Hello, Prachi
```

- **Multiline Strings**

If our string has multiple lines, we can create them like this:

```
a = """Lorem ipsum dolor sit amet,  
consectetur adipiscing elit,  
sed do eiusmod tempor incididunt  
ut labore et dolore magna aliqua."""  
print(a)
```





# Accessing Characters of a String

- In Python, string is like an array of characters. We can access parts of string by using its index which starts from 0.
- Square brackets can be used to access elements of the string.

```
print(name[0])  
print(name[1])
```

## Looping through the string

- We can loop through strings using a for loop like this:  
for character in name:  
    print(character)
- Above code prints all the characters in the string name one by one!



# String Slicing & Operations on String

## Length of a String:

We can find the length of a string using len() function.

Example:

```
fruit = "Mango"
```

```
len1 = len(fruit)
```

```
print("Mango is a", len1, "letter word.")
```

#Output: Mango is a 5 letter word.



## String as an array:

A string is essentially a sequence of characters also called an array. Thus we can access the elements of this array.

Example:

```
pie = "ApplePie"  
print(pie[:5])  
print(pie[6])           #returns character at specified index
```

#Output: Apple  
i

Note: This method of specifying the start and end index to specify a part of a string is called slicing.



# Slicing Example:

```
pie = "ApplePie"  
print(pie[:5])    #Slicing from Start  
print(pie[5:])    #Slicing till End  
print(pie[2:6])   #Slicing in between  
print(pie[-8:])   #Slicing using negative index
```

#Output:

```
Apple  
Pie  
pleP  
ApplePie
```



# Loop through a String:

Strings are arrays and arrays are iterable. Thus we can loop through strings.

Example:

```
alphabets = "ABCDE"  
for i in alphabets:  
    print(i)
```

Output:

A  
B  
C  
D  
E

# String Methods



- Strings are immutable. Python provides a set of built-in methods that we can use to alter and modify the strings.

## **upper() :**

The upper() method converts a string to upper case.

Example:

```
str1 = "AbcDEfghIJ"  
print(str1.upper())
```

Output:  
ABCDEFGHIJ

## **lower()**

The lower() method converts a string to lower case.

Example:

```
str1 = "AbcDEfghIJ"  
print(str1.lower())
```

Output:  
abcdefghij

## **strip() :**

The strip() method removes any white spaces before and after the string.

Example:

```
str2 = " Silver Spoon "  
print(str2.strip)
```

Output:  
Silver Spoon



### **rstrip() :**

the rstrip() removes any trailing characters. Example:

Example:

```
str3 = "Hello !!!"
```

```
print(str3.rstrip("!!!"))
```

Output:  
Hello

### **replace() :**

The replace() method replaces all occurrences of a string with another string. Example:

Example:

```
str2 = "Silver Spoon"
```

```
print(str2.replace("Sp", "M"))
```

Output:  
Silver Moon

### **split() :**

The split() method splits the given string at the specified instance and returns the separated strings as list items.

Example:

```
str2 = "Silver Spoon"
```

```
print(str2.split(" "))    #Splits the string at the whitespace " ".
```

Output:  
['Silver', 'Spoon']

There are various other string methods that we can use to modify our strings.



### **capitalize() :**

The capitalize() method turns only the first character of the string to uppercase and the rest other characters of the string are turned to lowercase. The string has no effect if the first character is already uppercase.

Example:

```
str1 = "hello"  
capStr1 = str1.capitalize()  
print(capStr1)  
str2 = "hello WorlD"  
capStr2 = str2.capitalize()  
print(capStr2)
```

Output:  
Hello  
Hello world

### **count() :**

The count() method returns the number of times the given value has occurred within the given string.

Example:

```
str2 = "Abracadabra"  
countStr = str2.count("a")  
print(countStr)
```

Output:  
4





### **endswith() :**

The endswith() method checks if the string ends with a given value. If yes then return True, else return False.

Example :

```
str1 = "Welcome to the Console !!!"  
print(str1.endswith("!!!"))
```

Output:  
True

We can even also check for a value in-between the string by providing start and end index positions.

Example:

```
str1 = "Welcome to the Console !!!"  
print(str1.endswith("to", 4, 10))
```

Output:  
True

### **find() :**

The find() method searches for the first occurrence of the given value and returns the index where it is present. If given value is absent from the string then return -1.

Example:

```
str1 = "He's name is Dan. He is an honest man."  
print(str1.find("is"))
```

Output:  
10

As we can see, this method is somewhat similar to the index() method. The major difference being that index() raises an exception if value is absent whereas find() does not.

Example:

```
str1 = "He's name is Dan. He is an honest man."  
print(str1.find("Daniel"))
```

Output:  
-1



## **index() :**

The index() method searches for the first occurrence of the given value and returns the index where it is present. If given value is absent from the string then raise an exception.

Example:

```
str1 = "He's name is Dan. Dan is an honest man."  
print(str1.index("Dan"))
```

Output:  
13

As we can see, this method is somewhat similar to the find() method. The major difference being that index() raises an exception if value is absent whereas find() does not.

Example:

```
str1 = "He's name is Dan. Dan is an honest man."  
print(str1.index("Daniel"))
```

Output:  
ValueError: substring not found

## **isalnum() :**

The isalnum() method returns True only if the entire string only consists of A-Z, a-z, 0-9. If any other characters or punctuations are present, then it returns False.

Example 1:

```
str1 = "WelcomeToTheConsole"  
print(str1.isalnum())
```

Output:  
True



### **isalpha() :**

The `isalpha()` method returns `True` only if the entire string only consists of A-Z, a-z. If any other characters or punctuations or numbers(0-9) are present, then it returns `False`.

Example :

```
str1 = "Welcome"  
print(str1.isalpha())
```

Output:  
True

### **islower() :**

The `islower()` method returns `True` if all the characters in the string are lower case, else it returns `False`.

Example:

```
str1 = "hello world"  
print(str1.islower())
```

Output:  
True

### **isprintable() :**

The `isprintable()` method returns `True` if all the values within the given string are printable, if not, then return `False`.

Example :

```
str1 = "We wish you a Merry Christmas"  
print(str1.isprintable())
```

Output:  
True

## **isspace() :**

The isspace() method returns True only and only if the string contains white spaces, else returns False.

Example:

```
str1 = "      "      #using Spacebar
print(str1.isspace())
str2 = "\t\t\t"      #using Tab
print(str2.isspace())
```

Output:  
True  
True



## **istitle() :**

The istitle() returns True only if the first letter of each word of the string is capitalized, else it returns False.

Example:

```
str1 = "World Health Organization"
print(str1.istitle())
```

Output:  
True

Example:

```
str2 = "To kill a Mocking bird"
print(str2.istitle())
```

Output:  
False

### **isupper() :**

The isupper() method returns True if all the characters in the string are upper case, else it returns False.

Example :

```
str1 = "WORLD HEALTH ORGANIZATION"  
print(str1.isupper())
```

Output:  
True



### **startswith() :**

The endswith() method checks if the string starts with a given value. If yes then return True, else return False.

Example :

```
str1 = "Python is a Interpreted Language"  
print(str1.startswith("Python"))
```

Output:  
True

### **swapcase() :**

The swapcase() method changes the character casing of the string. Upper case are converted to lower case and lower case to upper case.

Example:

```
str1 = "Python is a Interpreted Language"  
print(str1.swapcase())
```

Output:  
pYTHON IS A INTERPRETED IANGUAGE

### **title() :**

The title() method capitalizes each letter of the word within the string.

Example:

```
str1 = "His name is Dan. Dan is an honest man."  
print(str1.title())
```

Output:  
His Name Is Dan. Dan Is An Honest Man.



# MULTIPLE ASSIGNMENTS

Python supports simultaneous assignment to multiple variables. The syntax of multiple assignments is

*Var1, Var2, Var3, ..... = Exp1, Exp2, Exp3, ..... ExpN*

In the above syntax, Python simultaneously evaluates all the expressions on the right and assigns them to a corresponding variable on the left.

```
>>> P, Q = Q, P #Swap P with Q & Q with P
```



# eval() function

The full form of eval function is to evaluate. It takes a string as parameter and returns it as if it is a Python expression.

## **Example**

```
>>> X = eval('123')
>>> X
123
>>> print(type(X))
<class 'int'>
```

## **Apply eval() to input() Function**

In the previous section we learnt about the input() function in detail. We know that the input() function returns every input by the user as string, including numbers. By making use of eval() function, we can avoid specifying a particular type in front of input() function.

```
X = int (input('Enter the Number'))
can be written as:
X = eval(input('Enter the Number'))
```

With respect to the above statement, a programmer does not know what values a user can enter. He/she may enter a value of any type, i.e. int, float, string, complex etc. By making use of eval, Python automatically determines the type of value entered by the user.

# The ord and chr Functions



- As we know, a string is a sequence of characters. It can include both text and numbers. All these characters are stored in a computer as a sequence of 0s and 1s. Therefore, a process of mapping a character to its binary representation is called character encoding.
- There are different ways to encode a character. The encoding scheme decides the manner in which characters are encoded. The American Standard Code for Information Interchangeable ( ASCII) is one of the most popular encoding schemes. It is a 7-bit encoding scheme for representation of all lower and upper case letters, digits and punctuation marks. The ASCII uses numbers from 0 to 127 to represent all characters.
- Python uses the in-built function ord(ch) to return the ASCII value for a given character.

Example

```
>>> ord('A') #Returns ASCII value of Character 'A'
```

```
65
```

```
>>> ord('Z') #Returns ASCII Value of Character of 'Z'
```

```
90 >>>
```

```
ord('a') #Returns ASCII Value of Character of 'a'
```

```
97
```

- The chr(Code) returns the character corresponding to its code, i.e. the ASCII value.

Example

```
>>> chr(90)
```

```
'Z'
```

```
>>> chr(65)
```

```
'A'
```





# File Handling

## Opening a File

The open function creates a file object.

Before we can perform any operations on a file, we must first open it. Python provides the open() function to open a file. It takes two arguments: the name of the file and the mode in which the file should be opened. The mode can be 'r' for reading, 'w' for writing, or 'a' for appending.

Here's an example of how to open a file for reading:

```
f = open('myfile.txt', 'r')
```

By default, the open() function returns a file object that can be used to read from or write to the file, depending on the mode.

### **Example**

```
F1 = open ("Demo.txt","r") #Open File from Current Directory  
F2 = open ("c:\Hello.txt","r")
```

The above example opens a file named Hello.txt located at C: in read mode.



# Modes in file

---

There are various modes in which we can open files.

1. read (r): This mode opens the file for reading only and gives an error if the file does not exist. This is the default mode if no mode is passed as a parameter.
2. write (w): This mode opens the file for writing only and creates a new file if the file does not exist.
3. append (a): This mode opens the file for appending only and creates a new file if the file does not exist.
4. create (x): This mode creates a file and gives an error if the file already exists.
5. text (t): Apart from these modes we also need to specify how the file must be handled. t mode is used to handle text files. t refers to the text mode. There is no difference between r and rt or w and wt since text mode is the default. The default mode is 'r' (open for reading text, synonym of 'rt' ).
6. binary (b): used to handle binary files (images, pdfs, etc).

# Reading from a File

---

Once we have a file object, we can use various methods to read from the file.

The `read()` method reads the entire contents of the file and returns it as a string.

```
f = open('myfile.txt', 'r')
contents = f.read()
print(contents)
```



```
f = open('myfile.txt', 'r')  
line = f.readline()  
print(line)
```

## readlines() method

The readline() method reads a single line from the file. If we want to read multiple lines, we can use a loop.

```
f = open('myfile.txt', 'r')  
while True:  
    line = f.readline()  
    if not line:  
        break  
    print(line)
```

The readlines() method reads all the lines of the file and returns them as a list of strings.



# Writing to a File

---

To write to a file, we first need to open it in write mode.

```
f = open('myfile.txt', 'w')
```

We can then use the write() method to write to the file.

```
f = open('myfile.txt', 'w')  
f.write('Hello, world!')
```

Keep in mind that writing to a file will overwrite its contents. If you want to append to a file instead of overwriting it, you can open it in append mode.

```
f = open('myfile.txt', 'a')  
f.write('Hello, world!')
```



# Writing Numbers to a File

In the above program, we have seen that the `write ()` method is used to write a string to a file. However, if we try to write numbers to a file, the Python interpreter shows an error. The following program uses the `write` method to show the error generated by the Python interpreter upon execution.

The `write ()` method expects a string as an argument. Therefore, if we want to write other data types, such as integers or floating point numbers then the numbers must be first converted into strings before writing them to an output file. In order to read the numbers correctly, we need to separate them using special characters, such as " " (**space**) or '\n' (**new line**). Program 13.2 uses `str` method to convert numbers into strings and write numbers to an output file.

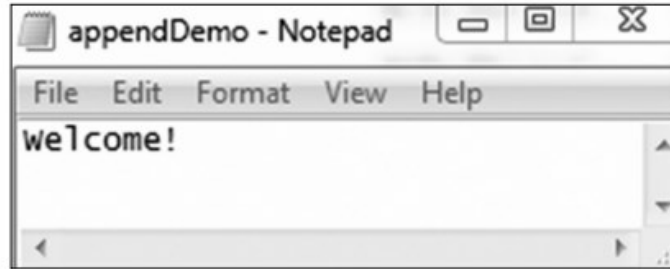


**Program: Write numbers from 1 to 20 to the output file WriteNumbers.txt.**

```
obj1 = open("WriteNumbers.txt","w") #Open File in Write mode
for x in range(1,21): # Iterates from 1 to 20
    x=str(x)           # Convert Number to String
    obj1.write(x)       # Write Number to a output file
    obj1.write(" ")     # Space to separate Numbers
obj1.close()           # Close File
```

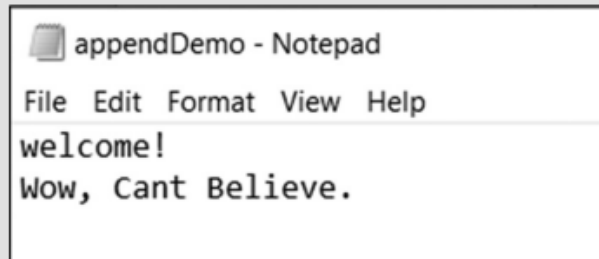
**Explanation:** The program opens a WriteNumbers.txt file in w mode, i.e. write mode. The for loop iterates 20 times to write numbers from 1 to 20 to the file. The numbers are converted into strings using the str method before being written to the file.

Write a program to append extra lines to a file name appendDemo.txt.



```
fp1=open('appendDemo.txt','a') # Open file in append file
fp1.write('\nWow, Cant Believe.')# Append contents to a file
fp1.close() #Close file
```

### Output







# Closing a File

---

It is important to close a file after you are done with it. This releases the resources used by the file and allows other programs to access it.

To close a file, you can use the `close()` method.

```
f = open('myfile.txt', 'r')  
# ... do something with the file  
f.close()
```

## The 'with' statement

---

Alternatively, you can use the `with` statement to automatically close the file after you are done with it.

```
with open('myfile.txt', 'r') as f:  
    # ... do something with the file
```



# writelines() method

The writelines() method in Python writes a sequence of strings to a file. The sequence can be any iterable object, such as a list or a tuple.

Here's an example of how to use the writelines() method:

```
f = open('myfile.txt', 'w')
lines = ['line 1\n', 'line 2\n', 'line 3\n']
f.writelines(lines)
f.close()
```

This will write the strings in the lines list to the file myfile.txt. The \n characters are used to add newline characters to the end of each string.

Keep in mind that the writelines() method does not add newline characters between the strings in the sequence. If you want to add newlines between the strings, you can use a loop to write each string separately:

# seek() and tell() functions

---

In Python, the `seek()` and `tell()` functions are used to work with file objects and their positions within a file. These functions are part of the built-in `io` module, which provides a consistent interface for reading and writing to various file-like objects, such as files, pipes, and in-memory buffers.

## seek() function

---

The `seek()` function allows you to move the current position within a file to a specific point. The position is specified in bytes, and you can move either forward or backward from the current position. For example:

```
with open('file.txt', 'r') as f:
    # Move to the 10th byte in the file
    f.seek(10)

    # Read the next 5 bytes
    data = f.read(5)
```



# tell() function

The tell() function returns the current position within the file, in bytes. This can be useful for keeping track of your location within the file or for seeking to a specific position relative to the current position. For example:

```
with open('file.txt', 'r') as f:
    # Read the first 10 bytes
    data = f.read(10)

    # Save the current position
    current_position = f.tell()

    # Seek to the saved position
    f.seek(current_position)
```



# truncate() function

When you open a file in Python using the open function, you can specify the mode in which you want to open the file. If you specify the mode as 'w' or 'a', the file is opened in write mode and you can write to the file. However, if you want to truncate the file to a specific size, you can use the truncate function.

Here is an example of how to use the truncate function:

```
with open('sample.txt', 'w') as f:  
    f.write('Hello World!')  
    f.truncate(5)  
  
with open('sample.txt', 'r') as f:  
    print(f.read())
```



# os Module in Python

The os module in Python is a built-in library that provides functions for interacting with the operating system. It allows you to perform a wide variety of tasks, such as reading and writing files, interacting with the file system, and running system commands.

Here are some common tasks you can perform with the os module:

**Reading and writing files:** The os module provides functions for opening, reading, and writing files. For example, to open a file for reading, you can use the open function:

## ***os.read()***

method in Python is used to *read* at most *n* bytes from the file associated with the given file descriptor.

**Syntax:** `os.read(fd, n)`

**Parameter:**

**fd:** A file descriptor representing the file to be read.

**n:** An integer value denoting the number of bytes to be read from the file associated with the given file descriptor fd.

**Return Type:** This method returns a bytestring which represents the bytes read from the file associated with the file descriptor fd.

```
import os

# Open the file in read-only mode
f = os.open("myfile.txt", os.O_RDONLY)

# Read the contents of the file
contents = os.read(f, 1024)

# Close the file
os.close(f)
```



To open a file for writing, you can use the `os.O_WRONLY` flag:

```
import os

# Open the file in write-only mode
f = os.open("myfile.txt", os.O_WRONLY)

# Write to the file
os.write(f, b"Hello, world!")

# Close the file
os.close(f)
```





# Interacting with the file system

The `os` module also provides functions for interacting with the file system. For example, you can use the `os.listdir` function to get a list of the files in a directory:

```
import os

# Get a list of the files in the current directory
files = os.listdir(".")
print(files) # Output: ['myfile.txt', 'otherfile.txt']
```

You can also use the `os.mkdir` function to create a new directory:

```
import os

# Create a new directory
os.mkdir("newdir")
```



# Reading and Writing CSV Files in Python

**CSV (Comma Separated Values)** format is the most common import and export format for spreadsheets and databases. It is one of the most common methods for exchanging data between applications and popular data format used in Data Science. It is supported by a wide range of applications. A CSV file stores tabular data in which each data field is separated by a *delimiter*(comma in most cases). To represent a CSV file, it must be saved with the **.csv** file extension.

## Reading from CSV file

Python contains a module called `csv` for the handling of CSV files. The reader class from the module is used for reading data from a CSV file. At first, the CSV file is opened using the `open()` method in 'r' mode(specifies read mode while opening a file) which returns the file object then it is read by using the `reader()` method of CSV module that returns the reader object that iterates throughout the lines in the specified CSV document.

### Example:

Consider the below CSV file –

|   | A     | B   | C     |
|---|-------|-----|-------|
| 1 | name  | age | grade |
| 2 | Steve | 13  | A     |
| 3 | John  | 14  | F     |
| 4 | Nancy | 14  | C     |
| 5 | Ravi  | 13  | B     |
| 6 |       |     |       |

```
import csv
```

```
# opening the CSV file
```

```
with open('Giants.csv', mode ='r')as file:
```

```
# reading the CSV file
```

```
csvFile = csv.reader(file)
```

```
# displaying the contents of the CSV file
```

```
for lines in csvFile:
```

```
    print(lines)
```

### Output:

```
[['Steve', 13, 'A'],
```

```
['John', 14, 'F'],
```

```
['Nancy', 14, 'C'],
```

```
['Ravi', 13, 'B']]
```



## Writing to CSV file

**csv.writer** class is used to insert data to the CSV file. This class returns a writer object which is responsible for converting the user's data into a delimited string. A CSV file object should be opened with `newline=""` otherwise, newline characters inside the quoted fields will not be interpreted correctly.

csv.writer class provides two methods for writing to CSV. They are `writerow()` and `writerows()`.

- **writerow():** This method writes a single row at a time. Field row can be written using this method.

**Syntax:**

```
writerow(fields)
```

- **writerows():** This method is used to write multiple rows at a time. This can be used to write rows list.

**Syntax:**

```
writerows(rows)
```

Example:  
# writing to  
CSV

```
import csv

# field names
fields = ['Name', 'Branch', 'Year', 'CGPA']

# data rows of csv file
rows = [ ['Nikhil', 'COE', '2', '9.0'],
         ['Sanchit', 'COE', '2', '9.1'],
         ['Aditya', 'IT', '2', '9.3'],
         ['Sagar', 'SE', '1', '9.5'],
         ['Prateek', 'MCE', '3', '7.8'],
         ['Sahil', 'EP', '2', '9.1']]

# name of csv file
filename = "university_records.csv"

# writing to csv file
with open(filename, 'w') as csvfile:
    # creating a csv writer object
    csvwriter = csv.writer(csvfile)

    # writing the fields
    csvwriter.writerow(fields)

    # writing the data rows
    csvwriter.writerows(rows)
```

Output:

|   | A       | B      | C    | D    | E |
|---|---------|--------|------|------|---|
| 1 | Name    | Branch | Year | CGPA |   |
| 2 | Nikhil  | COE    | 2    | 9    |   |
| 3 | Sanchit | COE    | 2    | 9.1  |   |
| 4 | Aditya  | IT     | 2    | 9.3  |   |
| 5 | Sagar   | SE     | 1    | 9.5  |   |
| 6 | Prateek | MCE    | 3    | 7.8  |   |
| 7 | Sahil   | EP     | 2    | 9.1  |   |
| 8 |         |        |      |      |   |
| 9 |         |        |      |      |   |





We can also write dictionary to the CSV file. For this the CSV module provides the **csv.DictWriter** class. This class returns a writer object which maps dictionaries onto output rows.

csv.DictWriter provides two methods for writing to CSV. They are:

- **writeheader():** `writeheader()` method simply writes the first row of your csv file using the pre-specified fieldnames.

**Syntax:**

```
writeheader()
```

- **writerows():** `writerows` method simply writes all the rows but in each row, it writes only the values(not keys).

**Syntax:**

```
writerows(mydict)
```



```
import csv
# my data rows as dictionary objects
mydict = [{'branch': 'COE', 'cgpa': '9.0', 'name': 'Nikhil', 'year': '2'},
          {'branch': 'COE', 'cgpa': '9.1', 'name': 'Sanchit', 'year': '2'},
          {'branch': 'IT', 'cgpa': '9.3', 'name': 'Aditya', 'year': '2'},
          {'branch': 'SE', 'cgpa': '9.5', 'name': 'Sagar', 'year': '1'},
          {'branch': 'MCE', 'cgpa': '7.8', 'name': 'Prateek', 'year': '3'},
          {'branch': 'EP', 'cgpa': '9.1', 'name': 'Sahil', 'year': '2'}]
```

```
# field names
```

```
fields = ['name', 'branch', 'year', 'cgpa']
```

```
# name of csv file
```

```
filename = "university_records.csv"
```

```
# writing to csv file
```

```
with open(filename, 'w') as csvfile:
```

```
    # creating a csv dict writer object
```

```
    writer = csv.DictWriter(csvfile, fieldnames = fields)
```

```
    # writing headers (field names)
```

```
    writer.writeheader()
```

```
    # writing data rows
```

```
    writer.writerows(mydict)
```

**Output:**

|   | A       | B      | C    | D    |  |
|---|---------|--------|------|------|--|
| 1 | Name    | Branch | Year | CGPA |  |
| 2 | Nikhil  | COE    | 2    | 9    |  |
| 3 | Sanchit | COE    | 2    | 9.1  |  |
| 4 | Aditya  | IT     | 2    | 9.3  |  |
| 5 | Sagar   | SE     | 1    | 9.5  |  |
| 6 | Prateek | MCE    | 3    | 7.8  |  |
| 7 | Sahil   | EP     | 2    | 9.1  |  |
| 8 |         |        |      |      |  |



# Chapter-4





# Python Functions

A function is a block of code that performs a specific task whenever it is called. In bigger programs, where we have large amounts of code, it is advisable to create or use existing functions that make the program flow organized and neat.

There are two types of functions:

1. Built-in functions
2. User-defined functions

## **Built-in functions:**

These functions are defined and pre-coded in python. Some examples of built-in functions are as follows: `min()`, `max()`, `len()`, `sum()`, `type()`, `range()`, `dict()`, `list()`, `tuple()`, `set()`, `print()`, etc.



# User-defined functions:

We can create functions to perform specific tasks as per our needs. Such functions are called user-defined functions.

## Syntax:

```
def function_name(parameters):  
    pass  
    # Code and Statements
```

- Create a function using the def keyword, followed by a function name, followed by a parenthesis (()) and a colon(:).
- Any parameters and arguments should be placed within the parentheses.
- Rules to naming function are similar to that of naming variables.
- Any statements and other code within the function should be indented.



The syntax for the Python function contains a header and body. The function header begins with the 'def' keyword. The def keyword signifies the beginning of the function's definition. The name of the function is followed by the def keyword. The function header may contain zero or more number of parameters. These parameters are called formal parameters. If a function contains more than one parameter then all the parameters are separated by commas. A function's body is a block of statements. The statements within the function's body define the actions that the function needs to perform.

Example: Write a program to create a function having a name, display. Print the message, "Welcome to Python Programming" inside the function

```
def Display():  
    print("Welcome to Python Programming ")  
Display()           #call function
```

### **Output**

```
Welcome to Python Programming
```



# Calling a function:

We call a function by giving the function name, followed by parameters (if any) in the parenthesis.

Example:

```
def name(fname, lname):  
    print("Hello,", fname, lname)  
  
name("Sam", "Wilson")
```

Output: Hello, Sam Wilson



# Function Arguments

There are four types of arguments that we can provide in a function:

- Default Arguments
- Keyword Arguments
- Variable length Arguments
- Required Arguments

## Default arguments:

We can provide a default value while creating a function. This way the function assumes a default value even if a value is not provided in the function call for that argument.

Example:

```
def name(fname, mname = "Jhon", lname = "Whatson"):  
    print("Hello,", fname, mname, lname)  
  
name("Amy")
```

**Output:**

Hello, Amy Jhon Whatson



## Keyword arguments:

We can provide arguments with key = value, this way the interpreter recognizes the arguments by the parameter name. Hence, the the order in which the arguments are passed does not matter.

Example:

```
def name(fname, mname, lname):  
    print("Hello,", fname, mname, lname)  
  
name(mname = "Peter", lname = "Wesker", fname = "Jade")
```

**Output:**  
**Hello, Jade Peter Wesker**



## Required arguments:

In case we don't pass the arguments with a key = value syntax, then it is necessary to pass the arguments in the correct positional order and the number of arguments passed should match with actual function definition.

Example 1: when number of arguments passed does not match to the actual function definition.

```
def name(fname, mname, lname):  
    print("Hello,", fname, mname, lname)  
  
name("Peter", "Quill")
```

### Output:

```
name("Peter", "Quill")\  
TypeError: name() missing 1 required  
positional argument: 'lname'
```



# Variable-length arguments:

Sometimes we may need to pass more arguments than those defined in the actual function. This can be done using variable-length arguments.

There are two ways to achieve this:

## Arbitrary Arguments:

While creating a function, pass a \* before the parameter name while defining the function. The function accesses the arguments by processing them in the form of tuple.

```
def name(*name):  
    print("Hello,", name[0], name[1], name[2])  
  
name("James", "Buchanan", "Barnes")
```

## Output:

Hello, James Buchanan Barnes





```
def average(*numbers):  
    print(type(numbers))  
    sum = 0  
    for i in numbers:  
        sum = sum + i  
    print("Average is: ", sum / len(numbers))
```



## Keyword Arbitrary Arguments:

While creating a function, pass a \* before the parameter name while defining the function. The function accesses the arguments by processing them in the form of dictionary. Example:

```
def name(**name):  
    print("Hello,", name["fname"], name["mname"], name["lname"])  
  
name(mname = "Buchanan", lname = "Barnes", fname = "James")
```

**Output:**

**Hello, James Buchanan Barnes**



# return Statement

The return statement is used to return the value of the expression back to the calling function.

Example:

```
def name(fname, mname, lname):  
    return "Hello, " + fname + " " + mname + " " + lname  
  
print(name("James", "Buchanan", "Barnes"))
```

**Output:**

**Hello, James Buchanan Barnes**



# Python Lists

- Lists are ordered collection of data items.
- They store multiple items in a single variable.
- List items are separated by commas and enclosed within square brackets [].
- Lists are changeable meaning we can alter them after creation.

a single list can contain items of different data types.

```
List = ["Abhijeet", 18, "FYBScIT", 9.8]
```



# List Index

Each item/element in a list has its own unique index. This index can be used to access any particular item from the list. The first item has index [0], second item has index [1], third item has index [2] and so on.

## Example:

```
colors = ["Red", "Green", "Blue", "Yellow", "Green"]  
#           [0]       [1]       [2]       [3]       [4]
```



# Accessing list items

We can access list items by using its index with the square bracket syntax []. For example colors[0] will give "Red", colors[1] will give "Green" and so on...

## Positive Indexing:

As we have seen that list items have index, as such we can access items using these indexes.

## Example:

```
colors = ["Red", "Green", "Blue", "Yellow", "Green"]  
#           [0]      [1]      [2]      [3]      [4]  
print(colors[2])  
print(colors[4])  
print(colors[0])
```

## Output:

Blue  
Green  
Red



## Negative Indexing:

Similar to positive indexing, negative indexing is also used to access items, but from the end of the list. The last item has index [-1], second last item has index [-2], third last item has index [-3] and so on.

### Example:

```
colors = ["Red", "Green", "Blue", "Yellow", "Green"]  
#           [-5]    [-4]    [-3]    [-2]    [-1]  
print(colors[-1])  
print(colors[-3])  
print(colors[-5])
```

Output:

```
Green  
Blue  
Red
```



# Check whether an item is present in the list?

We can check if a given item is present in the list. This is done using the **in** keyword.

```
colors = ["Red", "Green", "Blue", "Yellow", "Green"]
if "Yellow" in colors:
    print("Yellow is present.")
else:
    print("Yellow is absent.")
```





# Range of Index:

You can print a range of list items by specifying where you want to start, where do you want to end and if you want to skip elements in between the range.

## Syntax:

**listName[start : end : jumpIndex]**

Note: jump Index is optional.

```
animals = ["cat", "dog", "bat", "mouse", "pig", "horse", "donkey", "goat", "cow"]
print(animals[3:7])      #using positive indexes
print(animals[-7:-2])    #using negative indexes'
```

Here, we provide index of the element from where we want to start and the index of the element till which we want to print the values.

Note: The element of the end index provided will not be included.

```
['mouse', 'pig', 'horse', 'donkey']
['bat', 'mouse', 'pig', 'horse', 'donkey']
```



## Example: printing all element from a given index till the end

```
animals = ["cat", "dog", "bat", "mouse", "pig", "horse", "donkey", "goat", "cow"]  
print(animals[4:])      #using positive indexes  
print(animals[-4:])     #using negative indexes
```

## Output:

```
['pig', 'horse', 'donkey', 'goat', 'cow']  
['horse', 'donkey', 'goat', 'cow']
```



## Example: printing all elements from start to a given index

```
animals = ["cat", "dog", "bat", "mouse", "pig", "horse", "donkey", "goat", "cow"]  
print(animals[:6])      #using positive indexes  
print(animals[:-3])     #using negative indexes
```

### Output:

```
['cat', 'dog', 'bat', 'mouse', 'pig', 'horse']  
['cat', 'dog', 'bat', 'mouse', 'pig', 'horse']
```

When no start index is provided, the interpreter prints all the values from start up to the end index provided.



## Example: Printing alternate values

```
animals = ["cat", "dog", "bat", "mouse", "pig", "horse", "donkey", "goat", "cow"]  
print(animals[::2])           #using positive indexes  
print(animals[-8:-1:2]) #using negative indexes
```

## Output:

```
['cat', 'bat', 'pig', 'donkey', 'cow']  
['dog', 'mouse', 'horse', 'goat']
```



# List Comprehension

List comprehensions are used for creating new lists from other iterables like lists, tuples, dictionaries, sets, and even in arrays and strings.

## Syntax:

List = [Expression(item) for item in iterable if Condition]

Expression: It is the item which is being iterated.

Iterable: It can be list, tuples, dictionaries, sets, and even in arrays and strings.

Condition: Condition checks if the item should be added to the new list or not.



## Example 1: Accepts items with the small letter “o” in the new list

```
names = ["Milo", "Sarah", "Bruno", "Anastasia", "Rosa"]  
namesWith_O = [item for item in names if "o" in item]  
print(namesWith_O)
```

### Output:

```
['Milo', 'Bruno', 'Rosa']
```

## Example 2: Accepts items which have more than 4 letters

```
names = ["Milo", "Sarah", "Bruno", "Anastasia", "Rosa"]  
namesWith_0 = [item for item in names if (len(item) > 4)]  
print(namesWith_0)
```

## Output:

```
['Sarah', 'Bruno', 'Anastasia']
```



# List Methods

## **list.sort()**

This method sorts the list in ascending order. The original list is updated

### Example 1:

```
colors = ["voilet", "indigo", "blue", "green"]
colors.sort()
print(colors)

num = [4,2,5,3,6,1,2,1,2,8,9,7]
num.sort()
print(num)
```

### Output:

```
['blue', 'green', 'indigo', 'voilet']\n[1, 1, 2, 2, 2, 3, 4, 5, 6, 7, 8, 9]
```



## Example:

```
colors = ["voilet", "indigo", "blue", "green"]
colors.sort(reverse=True)
print(colors)

num = [4,2,5,3,6,1,2,1,2,8,9,7]
num.sort(reverse=True)
print(num)
```

## Output:

```
['voilet', 'indigo', 'green', 'blue']
[9, 8, 7, 6, 5, 4, 3, 2, 2, 2, 1, 1]
```



# reverse()

---

This method reverses the order of the list.

**Example:**

```
colors = ["voilet", "indigo", "blue", "green"]
colors.reverse()
print(colors)

num = [4,2,5,3,6,1,2,1,2,8,9,7]
num.reverse()
print(num)
```

**Output:**

```
['green', 'blue', 'indigo', 'voilet']
[7, 9, 8, 2, 1, 2, 1, 6, 3, 5, 2, 4]
```

# index()

---

This method returns the index of the first occurrence of the list item.

Example:

```
colors = ["voilet", "green", "indigo", "blue", "green"]
print(colors.index("green"))

num = [4,2,5,3,6,1,2,1,3,2,8,9,7]
print(num.index(3))
```

Output:

```
1
3
```



# count()

---

Returns the count of the number of items with the given value.

Example:

```
colors = ["voilet", "green", "indigo", "blue", "green"]  
print(colors.count("green"))
```

```
num = [4,2,5,3,6,1,2,1,3,2,8,9,7]
```

Output:

```
2
```

```
3
```

# copy()

Returns copy of the list. This can be done to perform operations on the list without modifying the original list.

Example:

```
colors = ["voilet", "green", "indigo", "blue"]
newlist = colors.copy()
print(colors)
print(newlist)
```

Output:

```
['voilet', 'green', 'indigo', 'blue']
['voilet', 'green', 'indigo', 'blue']
```



# append():

---

This method appends items to the end of the existing list.

Example:

```
colors = ["voilet", "indigo", "blue"]  
colors.append("green")  
print(colors)
```

Output:

```
['voilet', 'indigo', 'blue', 'green']
```



## insert():

This method inserts an item at the given index. User has to specify index and the item to be inserted within the insert() method.

Example:

```
colors = ["voilet", "indigo", "blue"]
#           [0]         [1]         [2]

colors.insert(1, "green")  #inserts item at index 1
# updated list: colors = ["voilet", "green", "indigo", "blue"]
#           indexes           [0]         [1]         [2]         [3]

print(colors)
```

Output:

```
['voilet', 'green', 'indigo', 'blue']
```

## extend():

---

This method adds an entire list or any other collection datatype (set, tuple, dictionary) to the existing list.

### Example 1:

```
#add a list to a list
colors = ["voilet", "indigo", "blue"]
rainbow = ["green", "yellow", "orange", "red"]
colors.extend(rainbow)
print(colors)
```

### Output:

```
['voilet', 'indigo', 'blue', 'green', 'yellow', 'orange', 'red']
```



# Concatenating two lists:

---

You can simply concatenate two lists to join two lists.

Example:

```
colors = ["voilet", "indigo", "blue", "green"]  
colors2 = ["yellow", "orange", "red"]  
print(colors + colors2)
```

Output:

```
['voilet', 'indigo', 'blue', 'green', 'yellow', 'orange', 'red']
```

**PROGRAM 8.1** | Write a program to create a list with elements 1,2,3,4 and 5. Display even elements of the list using list comprehension.

```
List1=[1,2,3,4,5]
print("Content of List1")
print(List1)
List1=[x for x in List1 if x%2==0]
print("Even elements from the List1")
print(List1)
```

### Output

```
Content of List1
[1, 2, 3, 4, 5]
Even elements from the List1
[2, 4]
```



# Python Tuples

Tuples are ordered collection of data items. They store multiple items in a single variable. Tuple items are separated by commas and enclosed within round brackets (). Tuples are unchangeable meaning we can not alter them after creation.

EXAMPLE:

```
details = ("Abhijeet", 18, "FYBScIT", 9.8)
print(details)
```

Output:

```
('Abhijeet', 18, 'FYBScIT', 9.8)
```



# Tuple Indexes

Like strings, lists, each item/element in a tuple also has its own unique index. This index can be used to access any particular item from the tuple. The first item has index [0], second item has index [1], third item has index [2] and so on.

## Example:

```
country = ("Spain", "Italy", "India",)
#           [0]      [1]      [2]
```

### I. Positive Indexing:

As we have seen that tuple items have index, as such we can access items using these indexes.

Example:

```
country = ("Spain", "Italy", "India",)
#           [0]      [1]      [2]
print(country[0])
print(country[1])
print(country[2])
```

Output:

```
Spain
Italy
India
```



## II. Negative Indexing:

Similar to positive indexing, negative indexing is also used to access items, but from the end of the tuple. The last item has index [-1], second last item has index [-2], third last item has index [-3] and so on.

### Example:

```
country = ("Spain", "Italy", "India", "England", "Germany")
#           [0]      [1]      [2]      [3]      [4]
print(country[-1]) # Similar to print(country[len(country) - 1])
print(country[-3])
print(country[-4])
```

### Output:

```
Germany
India
Italy
```



### III. Check for item:

We can check if a given item is present in the tuple. This is done using the **in** keyword.

```
country = ("Spain", "Italy", "India", "England", "Germany")
if "Russia" in country:
    print("Russia is present.")
else:
    print("Russia is absent.")
```

Output:

```
Russia is absent.
```

## IV. Range of Index:

You can print a range of tuple items by specifying where do you want to start, where do you want to end and if you want to skip elements in between the range.

### Syntax:

**Tuple [start : end : jumpIndex]**

Note: jump Index is optional.

```
animals = ("cat", "dog", "bat", "mouse", "pig", "horse", "donkey", "goat", "cow")  
print(animals[4:])      #using positive indexes  
print(animals[-4:])     #using negative indexes
```

```
('pig', 'horse', 'donkey', 'goat', 'cow')  
( 'horse', 'donkey', 'goat', 'cow')
```

```
print(animals[:6])      #using positive indexes  
print(animals[:-3])     #using negative indexes
```

```
('cat', 'dog', 'bat', 'mouse', 'pig', 'horse')  
( 'cat', 'dog', 'bat', 'mouse', 'pig', 'horse')
```

```
print(animals[::2])      #using positive indexes  
print(animals[-8:-1:2]) #using negative indexes
```

```
('cat', 'bat', 'pig', 'donkey', 'cow')  
( 'dog', 'mouse', 'horse', 'goat')
```



# Manipulating Tuples

Tuples are immutable, hence if you want to add, remove or change tuple items, then first you must convert the tuple to a list. Then perform operation on that list and convert it back to tuple.

Example:

```
countries = ("Spain", "Italy", "India", "England", "Germany")
temp = list(countries)
temp.append("Russia")       #add item
temp.pop(3)                 #remove item
temp[2] = "Finland"         #change item
countries = tuple(temp)
print(countries)
```

Output:

```
('Spain', 'Italy', 'Finland', 'Germany', 'Russia')
```

Thus, we convert the tuple to a list, manipulate items of the list using list methods, then convert list back to a tuple. However, we can directly concatenate two tuples without converting them to list.





## Example:

```
countries = ("Pakistan", "Afghanistan", "Bangladesh", "ShriLanka")
countries2 = ("Vietnam", "India", "China")
southEastAsia = countries + countries2
print(southEastAsia)
```

## Output:

```
('Pakistan', 'Afghanistan', 'Bangladesh', 'ShriLanka', 'Vietnam', 'India', 'China')
```



# Tuple methods

As tuple is immutable type of collection of elements it have limited built in methods.

## count() Method

---

The count() method of Tuple returns the number of times the given element appears in the tuple.

### Syntax:

```
tuple.count(element)
```

### Example

```
Tuple1 = (0, 1, 2, 3, 2, 3, 1, 3, 2)
res = Tuple1.count(3)
print('Count of 3 in Tuple1 is:', res)
```

### Output

3



# index() method

---

The Index() method returns the first occurrence of the given element from the tuple.

## Syntax:

```
tuple.index(element, start, end)
```

Note: This method raises a ValueError if the element is not found in the tuple.

## Example

```
tuple = (0, 1, 2, 3, 2, 3, 1, 3, 2)
res = tuple.index(3)
print('First occurrence of 3 is', res)
```

## Output

3



# Python Sets

Sets are unordered collection of data items. They store multiple items in a single variable. Set items are separated by commas and enclosed within curly brackets {}. Sets are unchangeable, meaning you cannot change items of the set once created. Sets do not contain duplicate items.

Example:

```
info = {"Carla", 19, False, 5.9, 19}
print(info)
```

Output:

```
{False, 19, 5.9, 'Carla'}
```

Here we see that the items of set occur in random order and hence they cannot be accessed using index numbers. Also sets do not allow duplicate values.



You can access items of set using a for loop.

**Example:**

```
info = {"Carla", 19, False, 5.9}
for item in info:
    print(item)
```

**Output:**

```
False
Carla
19
5.9
```



# Python Dictionaries/Dictionary Literals

Dictionaries are ordered collection of data items. They store multiple items in a single variable. Dictionary items are key-value pairs that are separated by commas and enclosed within curly brackets {}.

Example:

```
info = {'name':'Karan', 'age':19, 'eligible':True}
print(info)
```

Output:

```
{'name': 'Karan', 'age': 19, 'eligible': True}
```



# Accessing Dictionary items:

## I. Accessing single values:

Values in a dictionary can be accessed using keys. We can access dictionary values by mentioning keys either in square brackets or by using get method.

Example:

```
info = {'name':'Karan', 'age':19, 'eligible':True}
print(info['name'])
print(info.get('eligible'))
```

Output:

```
Karan
True
```



## II. Accessing multiple values:

We can print all the values in the dictionary using values() method.

Example:

```
info = {'name':'Karan', 'age':19, 'eligible':True}
print(info.values())
```

Output:

```
dict_values(['Karan', 19, True])
```





### III. Accessing keys:

We can print all the keys in the dictionary using keys() method.

Example:

```
info = {'name':'Karan', 'age':19, 'eligible':True}
print(info.keys())
```

Output:

```
dict_keys(['name', 'age', 'eligible'])
```



## IV. Accessing key-value pairs:

We can print all the key-value pairs in the dictionary using `items()` method.

Example:

```
info = {'name':'Karan', 'age':19, 'eligible':True}
print(info.items())
```

Output:

```
dict_items([('name', 'Karan'), ('age', 19), ('eligible', True)])
```



# String formatting in python

String formatting can be done in python using the format method.

```
txt = "For only {price:.2f} dollars!"  
print(txt.format(price = 49))
```

## f-strings in python

It is a new string formatting mechanism introduced by the PEP 498. It is also known as Literal String Interpolation or more commonly as F-strings (f character preceding the string literal). The primary focus of this mechanism is to make the interpolation easier.

When we prefix the string with the letter 'f', the string becomes the f-string itself. The f-string can be formatted in much same as the str.format() method. The f-string offers a convenient way to embed Python expression inside string literals for formatting.



```
name = 'Tushar'  
age = 23  
print(f"Hello, My name is {name} and I'm {age} years old.")
```

## Output:

---

```
Hello, My name is Tushar and I'm 23 years old.
```

In the above code, we have used the f-string to format the string. It evaluates at runtime; we can put all valid Python expressions in them.

We can use it in a single statement as well.

## Example

---

```
print(f"{2 * 30}")
```

## Output:

---

```
60
```



# Dictionary Methods

Dictionary uses several built-in methods for manipulation.

## update()

The update() method updates the value of the key provided to it if the item already exists in the dictionary, else it creates a new key-value pair.

Example:

```
info = {'name':'Karan', 'age':19, 'eligible':True}
print(info)
info.update({'age':20})
info.update({'DOB':2001})
print(info)
```

Output:

```
{'name': 'Karan', 'age': 19, 'eligible': True}
{'name': 'Karan', 'age': 20, 'eligible': True, 'DOB': 2001}
```



## Removing items from dictionary:

---

There are a few methods that we can use to remove items from dictionary.

### **clear():**

The clear() method removes all the items from the list.

### **Example:**

```
info = {'name':'Karan', 'age':19, 'eligible':True}
info.clear()
print(info)
```

### **Output:**

```
{}
```



## pop():

The pop() method removes the key-value pair whose key is passed as a parameter.

### Example:

```
info = {'name':'Karan', 'age':19, 'eligible':True}
info.pop('eligible')
print(info)
```

### Output:

```
{'name': 'Karan', 'age': 19}
```



## popitem():

The popitem() method removes the last key-value pair from the dictionary.

### Example:

```
info = {'name':'Karan', 'age':19, 'eligible':True, 'DOB':2003}  
info.popitem()  
print(info)
```

### Output:

```
{'name': 'Karan', 'age': 19, 'eligible': True}
```





## del:

we can also use the del keyword to remove a dictionary item.

**Example:**

```
info = {'name':'Karan', 'age':19, 'eligible':True, 'DOB':2003}  
del info['age']  
print(info)
```

**Output:**

```
{'name': 'Karan', 'eligible': True, 'DOB': 2003}
```

If key is not provided, then the del keyword will delete the dictionary entirely.



**PROGRAM 11.10** | Write a function histogram that takes string as parameter and generates a frequency of characters contained in it.

```
S = "AAPPLE"
```

The program should create a dictionary

```
D = {'A': 2, 'E': 1, 'P': 2, 'L': 1}
```



**PROGRAM 11.10** | Write a function histogram that takes string as parameter and generates a frequency of characters contained in it.

```
S = "AAPPLE"
```

The program should create a dictionary

```
D = {'A': 2, 'E': 1, 'P': 2, 'L': 1}
```

```
def Histogram(S):  
    D=dict() #Initially Create Empty Dictionary  
    for C in S:  
        if C not in D:  
            D[C] = 1  
        else:  
            D[C]=D[C]+1  
    return D  
H=Histogram("AAPPLE")  
print(H)
```



# How importing in python works

Importing in Python is the process of loading code from a Python module into the current script. This allows you to use the functions and variables defined in the module in your current script, as well as any additional modules that the imported module may depend on.

To import a module in Python, you use the import statement followed by the name of the module. For example, to import the math module, which contains a variety of mathematical functions, you would use the following statement:

```
import math
```

Once a module is imported, you can use any of the functions and variables defined in the module by using the dot notation. For example, to use the sqrt function from the math module, you would write:

```
import math

result = math.sqrt(9)
print(result) # Output: 3.0
```



# from keyword

You can also import specific functions or variables from a module using the from keyword. For example, to import only the sqrt function from the math module, you would write:

```
from math import sqrt

result = sqrt(9)
print(result) # Output: 3.0
```

You can also import multiple functions or variables at once by separating them with a comma:

```
from math import sqrt, pi

result = sqrt(9)
print(result) # Output: 3.0

print(pi) # Output: 3.141592653589793
```



# Importing everything

It's also possible to import all functions and variables from a module using the \* wildcard. However, this is generally not recommended as it can lead to confusion and make it harder to understand where specific functions and variables are coming from.

```
from math import *
```

Python also allows you to rename imported modules using the as keyword. This can be useful if you want to use a shorter or more descriptive name for a module, or if you want to avoid naming conflicts with other modules or variables in your code.

## The "as" keyword

```
import math as m

result = m.sqrt(9)
print(result) # Output: 3.0

print(m.pi) # Output: 3.141592653589793
```



# The dir function

Finally, Python has a built-in function called `dir` that you can use to view the names of all the functions and variables defined in a module. This can be helpful for exploring and understanding the contents of a new module.

```
import math  
  
print(dir(math))
```

This will output a list of all the names defined in the `math` module, including functions like `sqrt` and `pi`, as well as other variables and constants.

In summary, the `import` statement in Python allows you to access the functions and variables defined in a module from within your current script. You can import the entire module, specific functions or variables, or use the `*` wildcard to import everything. You can also use the `as` keyword to rename a module, and the `dir` function to view the contents of a module.



# Chapter-5





# Introduction to Object-oriented programming

In programming languages, mainly there are two approaches that are used to write program or code.

- 1). Procedural Programming
- 2). Object-Oriented Programming

The procedure we are following till now is the “Procedural Programming” approach.

The basic idea of object-oriented programming (OOP) in Python is to use classes and objects to represent real-world concepts and entities.



A **class** is a blueprint or template for creating objects. It defines the properties and methods that an object of that class will have. Properties are the data or state of an object, and methods are the actions or behaviors that an object can perform.

An **object** is an instance of a class, and it contains its own data and methods. For example, you could create a class called "Person" that has properties such as name and age, and methods such as speak() and walk(). Each instance of the Person class would be a unique object with its own name and age, but they would all have the same methods to speak and walk.



# Key features

- One of the key features of OOP in Python is **encapsulation**, which means that the internal state of an object is hidden and can only be accessed or modified through the object's methods. This helps to protect the object's data and prevent it from being modified in unexpected ways.
- Another key feature of OOP in Python is **inheritance**, which allows new classes to be created that inherit the properties and methods of an existing class. This allows for code reuse and makes it easy to create new classes that have similar functionality to existing classes.
- **Polymorphism** is also supported in Python, which means that objects of different classes can be treated as if they were objects of a common class. This allows for greater flexibility in code and makes it easier to write code that can work with multiple types of objects.

In summary, OOP in Python allows developers to model real-world concepts and entities using classes and objects, encapsulate data, reuse code through inheritance, and write more flexible code through polymorphism.

# Python Class and Objects



A class is a blueprint or a template for creating objects, providing initial values for state (member variables or attributes), and implementations of behavior (member functions or methods). The user-defined objects are created using the class keyword.

## Creating a Class:

---

Let us now create a class using the class keyword.

```
class Details:  
    name = "Rohan"  
    age = 20
```

## Creating an Object:

---

Object is the instance of the class used to access the properties of the class. Now let's create an object of the class.

## Example:

```
obj1 = Details()
```



Now we can print values:

## Example:

```
class Details:  
    name = "Rohan"  
    age = 20  
  
obj1 = Details()  
print(obj1.name)  
print(obj1.age)
```

## Output:

```
Rohan  
20
```



- Class - A class is a collection of objects. A class contains the blueprints or the prototype from which the objects are being created. It is a logical entity that contains some attributes and methods.

- Classes are created by keyword class.
- Attributes are the variables that belongs to a class.
- Attributes are always public and can be accessed using the dot (.) operator. Eg - Myclass, Myattribute





- Objects - Object is an entity that has a state and behavior associated with it. It may be any real-world object like a mouse, keyboard, chair, table, etc. Integers, strings, floating-point numbers even arrays, and dictionaries, are all objects.

An object consists of -

- State - It is represented by the attribute of an object. It also reflects the properties of an object.
- Behavior - It is represented by the method of an object. It also reflects the response of an object to other objects.
- Identity - It gives a unique name to an object & enables one object to interact with other objects.



# self parameter

The self parameter is a reference to the current instance of the class and is used to access variables that belongs to the class.

It must be provided as the extra parameter inside the method definition.

```
class Details:
    name = "Rohan"
    age = 20

    def desc(self):
        print("My name is", self.name, "and I'm", self.age, "years old.")

obj1 = Details()
obj1.desc()
```

## Output:

```
My name is Rohan and I'm 20 years old.
```





# Constructors

A constructor is a special method in a class used to create and initialize an object of a class. There are different types of constructors. Constructor is invoked automatically when an object of a class is created.

A constructor is a unique function that gets called automatically when an object is created of a class. The main purpose of a constructor is to initialize or assign values to the data members of that class. It cannot return any value other than None.

## Syntax of Python Constructor

---

```
def __init__(self):  
    # initializations
```

init is one of the reserved functions in Python. In Object Oriented Programming, it is known as a constructor.

### Types of Constructors in Python

1. Parameterized Constructor
2. Default Constructor



## Parameterized Constructor in Python

When the constructor accepts arguments along with self, it is known as parameterized constructor. These arguments can be used inside the class to assign the values to the data members.

Example:

```
class Details:
    def __init__(self, animal, group):
        self.animal = animal
        self.group = group

obj1 = Details("Crab", "Crustaceans")
print(obj1.animal, "belongs to the", obj1.group, "group.")
```

Output:

```
Crab belongs to the Crustaceans group.
```



## Default Constructor in Python

When the constructor doesn't accept any arguments from the object and has only one argument, self, in the constructor, it is known as a Default constructor.

### Example:

```
class Details:  
    def __init__(self):  
        print("animal Crab belongs to Crustaceans group")  
obj1=Details()
```

### Output:

```
animal Crab belongs to Crustaceans group
```



# Inheritance in python

When a class derives from another class. The child class will inherit all the public and protected properties and methods from the parent class. In addition, it can have its own properties and methods, this is called as inheritance.

## Python Inheritance Syntax

```
class BaseClass:  
    Body of base class  
class DerivedClass(BaseClass):  
    Body of derived class
```

Derived class inherits features from the base class where new features can be added to it. This results in re-usability of code.

### Types of inheritance:

1. Single inheritance
2. Multiple inheritance
3. Multilevel inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance



## Single Inheritance:

Single inheritance enables a derived class to inherit properties from a single parent class, thus enabling code reusability and the addition of new features to existing code.

### Example:

```
class Parent:
    def func1(self):
        print("This function is in parent class.")

class Child(Parent):
    def func2(self):
        print("This function is in child class.")

object = Child()
object.func1()
object.func2()
```

### Output:

```
This function is in parent class.
This function is in child class.
```



## Multiple Inheritance:

When a class can be derived from more than one base class this type of inheritance is called multiple inheritances. In multiple inheritances, all the features of the base classes are inherited into the derived class.

```
class Mother:
    mothername = ""

    def mother(self):
        print(self.mothername)

class Father:
    fathername = ""

    def father(self):
        print(self.fathername)

class Son(Mother, Father):
    def parents(self):
        print("Father name is :", self.fathername)
        print("Mother :", self.mothername)

s1 = Son()
s1.fathername = "Mommy"
s1.mothername = "Daddy"
s1.parents()
```

### Output:

```
Father name is : Mommy
Mother name is : Daddy
```



## Multilevel Inheritance :

In multilevel inheritance, features of the base class and the derived class are further inherited into the new derived class. This is similar to a relationship representing a child and a grandfather.

```
class Grandfather:

    def __init__(self, grandfathername):
        self.grandfathername = grandfathername

class Father(Grandfather):
    def __init__(self, fathername, grandfathername):
        self.fathername = fathername
        Grandfather.__init__(self, grandfathername)
class Son(Father):
    def __init__(self, sonname, fathername, grandfathername):
        self.sonname = sonname
        Father.__init__(self, fathername, grandfathername)

    def print_name(self):
        print('Grandfather name :', self.grandfathername)
        print("Father name :", self.fathername)
        print("Son name :", self.sonname)
s1 = Son('Prince', 'Rampal', 'Lal mani')
print(s1.grandfathername)
s1.print_name()
```

### Output:

```
George
Grandfather name : George
Father name : Philip
Son name : Charles
```



## Hierarchical Inheritance:

When more than one derived class are created from a single base this type of inheritance is called hierarchical inheritance. In this program, we have a parent (base) class and two child (derived) classes.

### Example:

```
class Parent:
    def func1(self):
        print("This function is in parent class.")

class Child1(Parent):
    def func2(self):
        print("This function is in child 1.")

class Child2(Parent):
    def func3(self):
        print("This function is in child 2.")

object1 = Child1()
object2 = Child2()
object1.func1()
object1.func2()
object2.func1()
object2.func3()
```

### Output:

```
This function is in parent class.
This function is in child 1.
This function is in parent class.
This function is in child 2.
```





## Hybrid Inheritance:

Inheritance consisting of multiple types of inheritance is called hybrid inheritance.

```
class School:
    def func1(self):
        print("This function is in school.")

class Student1(School):
    def func2(self):
        print("This function is in student 1. ")

class Student2(School):
    def func3(self):
        print("This function is in student 2.")

class Student3(Student1, School):
    def func4(self):
        print("This function is in student 3.")

object = Student3()
object.func1()
object.func2()
```

### Output:

```
This function is in school.
This function is in student 1.
```



# Method Overriding in Python

Method overriding is a powerful feature in object-oriented programming that allows you to redefine a method in a derived class. The method in the derived class is said to override the method in the base class. When you create an instance of the derived class and call the overridden method, the version of the method in the derived class is executed, rather than the version in the base class.

In Python, method overriding is a way to customize the behavior of a class based on its specific needs. For example, consider the following base class:

```
class Shape:
    def area(self):
        pass
```

In this base class, the area method is defined, but does not have any implementation. If you want to create a derived class that represents a circle, you can override the area method and provide an implementation that calculates the area of a circle:

```
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius
```

In this example, the Circle class inherits from the Shape class, and overrides the area method. The new implementation of the area method calculates the area of a circle, based on its radius.



It's important to note that when you override a method, the new implementation must have the same method signature as the original method. This means that the number and type of arguments, as well as the return type, must be the same.

Another way to customize the behavior of a class is to call the base class method from the derived class method. To do this, you can use the super function. The super function allows you to call the base class method from the derived class method, and can be useful when you want to extend the behavior of the base class method, rather than replace it.

For example, consider the following base class:

```
class Shape:
    def area(self):
        print("Calculating area...")
```

In this base class, the area method prints a message indicating that the area is being calculated. If you want to create a derived class that represents a circle, and you also want to print a message indicating the type of shape, you can use the super function to call the base class method, and add your own message:

```
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        print("Calculating area of a circle...")
        super().area()
        return 3.14 * self.radius * self.radius
```

In this example, the Circle class overrides the area method, and calls the base class method using the super function. This allows you to extend the behavior of the base class method, while still maintaining its original behavior.



# Operator Overloading in Python

Operator Overloading is a feature in Python that allows developers to redefine the behavior of mathematical and comparison operators for custom data types. This means that you can use the standard mathematical operators (+, -, \*, /, etc.) and comparison operators (>, <, ==, etc.) in your own classes, just as you would for built-in data types like int, float, and str.

## Why do we need operator overloading?

Operator overloading allows you to create more readable and intuitive code. For instance, consider a custom class that represents a point in 2D space. You could define a method called 'add' to add two points together, but using the + operator makes the code more concise and readable:

```
p1 = Point(1, 2)
p2 = Point(3, 4)
p3 = p1 + p2
print(p3.x, p3.y) # prints 4, 6
```



## How to overload an operator in Python?

You can overload an operator in Python by defining special methods in your class. These methods are identified by their names, which start and end with double underscores (\_\_). Here are some of the most commonly overloaded operators and their corresponding special methods:

```
+ : __add__  
- : __sub__  
* : __mul__  
/ : __truediv__  
< : __lt__  
> : __gt__  
== : __eq__
```

For example, if you want to overload the + operator to add two instances of a custom class, you would define the **add** method:

```
class Point:  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y  
  
    def __add__(self, other):  
        return Point(self.x + other.x, self.y + other.y)
```

It's important to note that operator overloading is not limited to the built-in operators, you can overload any user-defined operator as well.



# Exception Handling

---

Exception handling is the process of responding to unwanted or unexpected events when a computer program runs. Exception handling deals with these events to avoid the program or system crashing, and without this process, exceptions would disrupt the normal operation of a program.

## Exceptions in Python

---

Python has many built-in exceptions that are raised when your program encounters an error (something in the program goes wrong).

When these exceptions occur, the Python interpreter stops the current process and passes it to the calling process until it is handled. If not handled, the program will crash.

## Python try...except

---

try..... except blocks are used in python to handle errors and exceptions. The code in try block runs when there is no error. If the try block catches the error, then the except block is executed.

# Syntax:

---



```
try:
    #statements which could generate
    #exception
except:
    #Soloution of generated exception
```

# Example:

---

```
try:
    num = int(input("Enter an integer: "))
except ValueError:
    print("Number entered is not an integer.")
```

# Output:

---

```
Enter an integer: 6.022
Number entered is not an integer.
```



# Finally Clause

The finally code block is also a part of exception handling. When we handle exception using the try and except block, we can include a finally block at the end. The finally block is always executed, so it is generally used for doing the concluding tasks like closing file resources or closing database connection or may be ending the program execution with a delightful message.

## Syntax:

---

```
try:
    #statements which could generate
    #exception
except:
    #solution of generated exception
finally:
    #block of code which is going to
    #execute in any situation
```

The finally block is executed irrespective of the outcome of try.....except.....else blocks  
One of the important use cases of finally block is in a function which returns a value.





## Example:

---

```
try:
    num = int(input("Enter an integer: "))
except ValueError:
    print("Number entered is not an integer.")
else:
    print("Integer Accepted.")
finally:
    print("This block is always executed.")
```

### Output 1:

---

```
Enter an integer: 19
Integer Accepted.
This block is always executed.
```

### Output 2:

---

```
Enter an integer: 3.142
Number entered is not an integer.
This block is always executed.
```



```
def func1():  
    try:  
        l = [1, 5, 6, 7]  
        i = int(input("Enter the index: "))  
        print(l[i])  
        return 1  
    except:  
        print("Some error occurred")  
        return 0  
  
    finally:  
        print("I am always executed")  
        # print("I am always executed")  
  
x = func1()  
print(x)
```



# Raising Custom errors

In python, we can raise custom errors by using the `raise` keyword.

```
salary = int(input("Enter salary amount: "))  
if not 2000 < salary < 5000:  
    raise ValueError("Not a valid salary")
```

In the previous sections, we learned about different built-in exceptions in Python and why it is important to handle exceptions. However, sometimes we may need to create our own custom exceptions that serve our purpose.



# Defining Custom Exceptions

In Python, we can define custom exceptions by creating a new class that is derived from the built-in Exception class.

Here's the syntax to define custom exceptions:

```
class CustomError(Exception):  
    # code ...  
    pass  
  
try:  
    # code ...  
  
except CustomError:  
    # code...
```

This is useful because sometimes we might want to do something when a particular exception is raised. For example, sending an error report to the admin, calling an api, etc.